

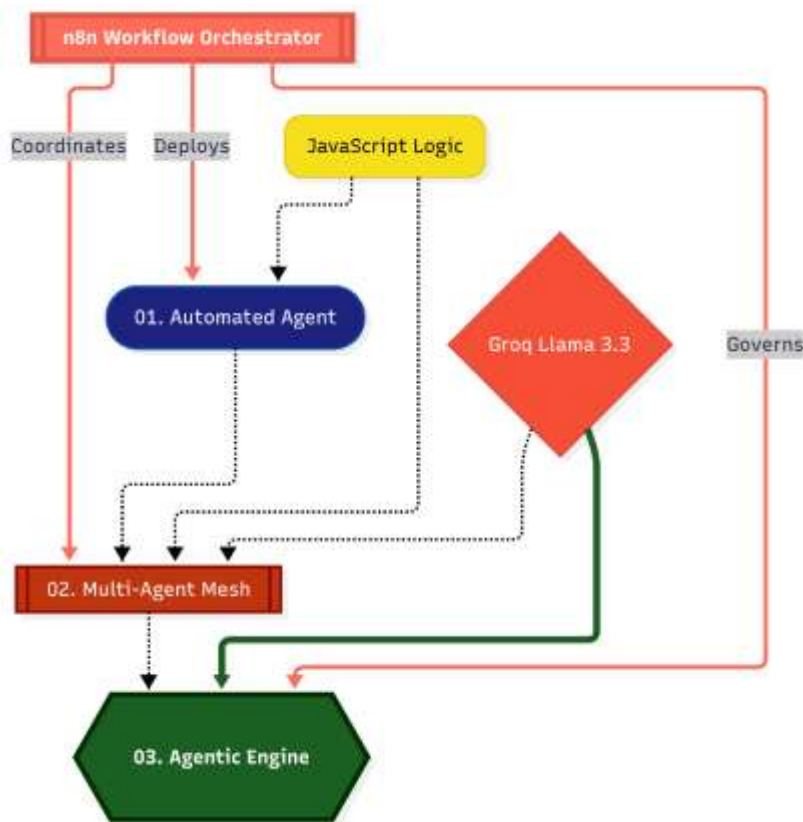
# AI-Driven Public Transport Operations Decision Agent

## Final Project Report

n8n + Groq LLM + JavaScript | 12 Functions | 4 Sub-Agents

Version 1.5 | April 24, 2026

NexusTransport AI | Muhammad Mobin



# Table of Contents

---

|                                                                |    |
|----------------------------------------------------------------|----|
| Table of Contents .....                                        | 2  |
| Executive Summary .....                                        | 6  |
| Development Progress Overview: From Concept to Deployment..... | 7  |
| 1. Introduction.....                                           | 8  |
| 1.1 Project Scope and Objectives.....                          | 8  |
| 1.2 System Summary Table .....                                 | 8  |
| 2. System Overview .....                                       | 9  |
| 2.1 Stakeholders.....                                          | 9  |
| 3. Requirements Specification .....                            | 10 |
| 3.1 Functional Requirements .....                              | 10 |
| 3.2 Non-Functional Requirements .....                          | 11 |
| 3.3 Use Cases.....                                             | 11 |
| UC01: Automated Route Optimization Cycle.....                  | 11 |
| UC02: Human Escalation on Low Confidence .....                 | 12 |
| UC03: High-Risk Congestion Response .....                      | 12 |
| 3.4 Constraints and Assumptions.....                           | 12 |
| Constraints .....                                              | 12 |
| 4. Design Techniques.....                                      | 13 |
| 4.1 Context Diagram.....                                       | 13 |
| 4.2 Layered Architecture Pattern Diagram .....                 | 13 |
| 4.3 Activity Diagram: Access Attempt.....                      | 14 |
| 4.4 Model-View-Controller Diagram .....                        | 14 |
| 4.5 Use Case Diagram.....                                      | 15 |
| 4.6 Use Case Tables.....                                       | 15 |
| 4.7 Sequence Diagram: Adding and Deleting a User .....         | 16 |
| 4.8 Activity Diagram: System Communications .....              | 17 |
| 4.9 Sequence Diagram: User Request.....                        | 18 |
| 5. Implementation.....                                         | 18 |
| 5.1 System Architecture Overview.....                          | 18 |
| 5.2 Trigger and Input Handling.....                            | 19 |
| 5.3 Data Ingestion Layer.....                                  | 19 |
| 5.4 Data Processing Layer .....                                | 19 |
| 5.4.1 Congestion Detection (Fn3 and Fn4).....                  | 19 |

|                                                     |    |
|-----------------------------------------------------|----|
| 5.4.2 Risk Classification (Fn6)                     | 19 |
| 5.4.3 Anomaly Detection (Fn5)                       | 19 |
| 5.4.4 Frequency Adjustment (Fn8)                    | 20 |
| 5.5 Multi-Agent AI Layer                            | 20 |
| 5.5.1 Main Coordinator Agent                        | 20 |
| 5.5.2 Specialized Sub-Agents                        | 20 |
| 5.5.3 Decision and Aggregation Layer                | 21 |
| 5.5.4 Action Layer                                  | 21 |
| 5.6 Traffic Index Calculation                       | 21 |
| 5.7 Output Layer                                    | 21 |
| 5.8 Memory Management                               | 21 |
| 5.9 Simplified Simulation Flow                      | 21 |
| 5.10 Multi-Agent Orchestration Workflow             | 22 |
| 5.11 Fully Agentic AI Workflow                      | 22 |
| 5.11.1 Key Architectural Upgrades                   | 23 |
| 5.11.2 Agentic Upgrade Path                         | 23 |
| 5.12 Implementation Strengths and Improvement Areas | 24 |
| 6. Build and Demonstration Record                   | 24 |
| 6.1 Basic AI Agent                                  | 24 |
| 6.1.1 Research                                      | 24 |
| 6.1.2 Build                                         | 24 |
| 6.1.3 Demonstration                                 | 25 |
| 6.2 AI Multi-Agent System                           | 26 |
| 6.2.1 Research                                      | 26 |
| 6.2.2 Build                                         | 27 |
| 6.2.3 Demonstration                                 | 28 |
| 6.3 Fully Agentic AI Workflow                       | 29 |
| 6.3.1 Research                                      | 29 |
| 6.3.2 Build                                         | 29 |
| 6.3.3 Demonstration                                 | 30 |
| 6.4 Mobile Risk Alert Application                   | 31 |
| 6.4.1 Application Design                            | 31 |
| 6.5 Intelligence Analytics Dashboard Website        | 33 |
| 6.5.1 Key Features                                  | 33 |
| 6.6 NexusTransport AI — Startup Concept             | 35 |

|                                                     |    |
|-----------------------------------------------------|----|
| 6.6.1 Business Model .....                          | 35 |
| 6.6.2 Competitive Advantage.....                    | 35 |
| 7. Testing and Debugging.....                       | 36 |
| 7.1 Testing Overview.....                           | 36 |
| 7.2 Functional Testing (Fn1 through Fn12) .....     | 36 |
| 7.3 Unit Testing: P-1 Test Plan .....               | 37 |
| 7.4 Subsystem Testing.....                          | 37 |
| 7.5 Integration Testing: P-2 Integration Plan ..... | 37 |
| 7.6 Strategic Agent Performance Matrix.....         | 37 |
| 7.7 Escalation and Safety Testing.....              | 37 |
| 7.8 System and Non-Functional Testing.....          | 38 |
| 8. Cybersecurity Specification .....                | 38 |
| 8.1 Security Architecture Overview .....            | 38 |
| 8.2 Layer 1: Ingress and API Security .....         | 38 |
| 8.3 Layer 2: Prompt Injection Defense.....          | 38 |
| 8.4 Layer 3: SA4 Compliance and Audit .....         | 39 |
| 8.5 Critical Escalation Security Threshold .....    | 39 |
| 9. Ethical Framework.....                           | 39 |
| 9.1 Ethical Governance Overview .....               | 39 |
| 9.2 Algorithmic Fairness and Bias Mitigation.....   | 40 |
| 9.3 Transparency and the Right to Explanation ..... | 40 |
| 9.4 Data Ethics and Passenger Privacy .....         | 40 |
| 9.5 Ethical Escalation Protocol .....               | 40 |
| 10. Post-Implementation Review .....                | 41 |
| 10.1 Review Objectives .....                        | 41 |
| 10.2 Performance Benchmarking.....                  | 41 |
| 10.3 Technical Debt and Memory Review .....         | 42 |
| 10.4 Strategic Agent Efficacy Audit .....           | 42 |
| 10.5 Future Roadmap.....                            | 42 |
| 11. COCOMO Cost Estimation .....                    | 42 |
| 11.1 Inputs and Cost Drivers .....                  | 42 |
| 11.2 Basic COCOMO Calculations.....                 | 43 |
| 11.3 CMM Level Impact and Final Estimates .....     | 43 |
| 11.4 Cost Summary at CMM Level 2 .....              | 43 |
| 12. Project Schedule and Activity Network.....      | 45 |

|                                                    |    |
|----------------------------------------------------|----|
| 12.1 Activity Table .....                          | 45 |
| 12.2 Activity Network Diagram .....                | 46 |
| 12.3 Gantt Chart.....                              | 47 |
| 13. Physical Robot Design .....                    | 48 |
| 13.1 Main Objectives and Project Learnings.....    | 48 |
| 13.2 Building Blocks.....                          | 48 |
| Transmitter and Base Station.....                  | 48 |
| Receiver and Robot Vehicle .....                   | 48 |
| 13.3 Agent Function to Robot Hardware Mapping..... | 49 |
| 13.4 Block Diagrams.....                           | 50 |
| 14. Mobile Application Design .....                | 52 |
| 14.1 Application Architecture .....                | 52 |
| 14.2 Data Integration and Logic .....              | 52 |
| 14.3 Functional Simulation Workflow.....           | 53 |
| 14.4 Scaling the Prototype .....                   | 53 |
| 15. Intelligence Dashboard .....                   | 54 |
| 15.1 Key Features.....                             | 54 |
| 15.2 Technical Stack .....                         | 54 |
| 16. Network and Telecom Reference Mapping .....    | 55 |
| 16.1 Network Diagram Analysis .....                | 55 |
| 16.2 Conceptual Mapping to Multi-Agent System..... | 55 |
| 17. Conclusion .....                               | 56 |
| 18. Project Activity Log.....                      | 57 |
| 19. References.....                                | 58 |
| Data Sources and API Providers .....               | 58 |
| Frameworks and Software Environments.....          | 58 |
| Hardware and Computational Architecture.....       | 58 |
| Security and Ethical Standards.....                | 59 |
| Project Management and Methodology.....            | 59 |
| Document Version History .....                     | 59 |

## Executive Summary

This report presents the complete design, implementation, and evaluation of the AI-Driven Public Transport Operations Decision Agent, a production-oriented multi-agent system built to optimize real-time transit operations across urban bus networks. Developed on the n8n workflow automation platform and powered by the Groq Llama-3.3-70b large language model, the system monitors 228 vehicles across 85 routes in the MBTA Boston network, executing a full decision cycle every two minutes.

The system is organized around a central Main Coordinator that delegates tasks to four specialized sub-agents: SA1 handles risk analysis and congestion detection, SA2 monitors anomalies, SA3 generates rerouting and frequency decisions, and SA4 manages audit logging, maintenance alerts, and the critical-exception escalation circuit. Together these agents execute 12 discrete functions from raw API ingestion through to structured output delivery.

A confidence-based escalation mechanism ensures that decisions reaching the autonomous action layer carry a confidence score of at least 0.6. Below that threshold, the system suspends autonomous action and immediately notifies a human dispatcher by email. The AI operates as the sole decision-maker under all routine and complex conditions, with human contact reserved strictly for situations where operational confidence collapses below the defined threshold.

The report covers the full project lifecycle: requirements specification, system design, implementation details, testing procedures, cybersecurity controls, ethical governance, post-implementation review, project cost estimation, physical robot design, mobile application design, dashboard architecture, and a telecom reference mapping. A development activity log and complete references are included at the end.

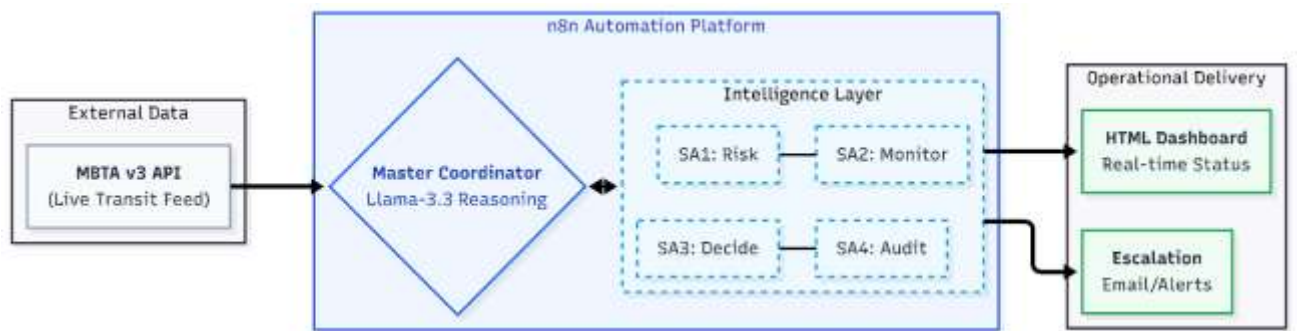


Figure 1. System Overview

## Development Progress Overview: From Concept to Deployment

---

1. Project Inception & API Research — Defined project goals for real-time AI transit monitoring, evaluated the MBTA v3 API as the live data source, and selected n8n and Groq (Llama-3.3-70b) as the core platform and LLM provider.
2. Basic Single-Agent Build — Implemented the foundational n8n workflow with a Schedule Trigger firing every two minutes, a single Groq LLM agent processing live MBTA vehicle data, and an HTML dashboard outputting Low, Medium, or High risk classifications across Fn1–Fn6.
3. Multi-Agent Architecture Design — Defined the four specialized sub-agent roles (SA1 Risk Analysis, SA2 Anomaly Detection, SA3 Optimization, SA4 Execution and Audit), established the confidence-based escalation threshold at 0.6, and designed the hub-and-spoke coordination model under the Main Coordinator Brain.
4. Full 12-Function Pipeline Implementation — Built and connected all 12 discrete JavaScript toolCode functions across SA1–SA4, covering data ingestion, normalization, congestion detection, delay assessment, anomaly monitoring, risk classification, rerouting decisions, frequency adjustments, maintenance alerts, escalation handling, audit logging, and effectiveness scoring.
5. Testing and Debugging — Conducted functional testing across all 12 functions, unit testing via the P-1 test plan, subsystem testing with stubs, and integration testing of the full multi-agent chain; validated the confidence switch escalation path using a forced low-confidence payload and confirmed correct email routing behavior.
6. Cybersecurity and Prompt Injection Hardening — Implemented layered security including API key encryption via n8n credentials, a whitelist-based data normalization filter in Fn2, negative constraint guardrails in the Main Coordinator system prompt, and SA4 audit logging for non-repudiation of all AI-generated actions.
7. Ethical Framework and Bias Mitigation — Integrated route-agnostic mathematical thresholds in SA1 to prevent algorithmic bias toward high-density corridors, added persistent underservice monitoring in SA2, and embedded plain-language reasoning fields in all outputs to satisfy transparency and explainability requirements.
8. Fully Agentic Upgrade Design — Designed the Version 2.0 upgrade path replacing fixed toolCode chains with ReAct-loop agents, added LangGraph stateful cycles for self-correction, implemented strategic Wait nodes for token rate management, and upgraded the binary confidence switch to a nuanced Escalation Router.
9. Supporting Deliverables — Developed and demonstrated five parallel deliverables: a mobile risk alert application (MIT App Inventor), an analytics dashboard website (HTML5, Tailwind CSS, Chart.js, Firebase), a physical robot design mapping all 12 functions to embedded hardware, a COCOMO cost estimate projecting \$103,440 at CMM Level 2, and the NexusTransport AI startup concept and business model.
10. Final Review and Documentation — Completed post-implementation review confirming a 45-second average decision cycle within the 120-second operational window and a 15% escalation rate; produced the full project report across 19 sections covering all lifecycle phases from requirements through conclusion, activity log, and references.

# 1. Introduction

The rapid urbanization of modern metropolitan areas has placed unprecedented strain on public transportation networks, leading to increased congestion, unpredictable delays, and operational inefficiencies. Traditional transit management often relies on manual dispatching and reactive problem-solving, which struggles to keep pace with the high-velocity data generated by modern fleet sensors. This project introduces the AI-Driven Public Transport Operations Decision Agent, an autonomous, multi-agent system designed to transform raw transit data into actionable operational intelligence.

Built on the n8n orchestration platform and powered by the Groq Llama-3.3-70b large language model, the system represents a departure from static, rule-based automation. Instead, it employs a "Brain" architecture consisting of a Main Coordinator and four specialized Strategic Agents (SA1 through SA4). These agents work in a synchronized chain to ingest live data from the MBTA v3 API, analyze traffic indices, monitor for anomalies, and execute rerouting decisions every two minutes. By distributing complex cognitive tasks such as risk assessment and ethical load balancing across dedicated AI roles, the system achieves a level of situational awareness that mirrors a highly experienced human dispatcher at a fraction of the response time.

The primary objective of this project is to demonstrate that agentic AI can manage critical public infrastructure with high reliability. Through the implementation of a confidence-based escalation system, the project establishes a fully autonomous operational framework, where the AI independently handles the entire decision pipeline. Human involvement is a last resort, triggered only when the system's own confidence measure signals an unresolvable ambiguity.

## 1.1 Project Scope and Objectives

The scope of this implementation covers the end-to-end lifecycle of a transit decision, from the initial API trigger to the final generation of an HTML dashboard or emergency email alert. The key objectives include:

- **Real-Time Responsiveness:** maintaining a strict two-minute operational cycle to ensure decisions reflect current road conditions.
- **Modular Intelligence:** utilizing a multi-agent framework to separate data cleaning, risk analysis, and decision-making for better debuggability and scalability.
- **Operational Safety:** implementing a critical-escalation safety circuit that activates only when AI confidence falls below 60 percent, a rare condition treated as a system-level exception rather than a routine operating mode.
- **Scalability:** designing a workflow that can expand from monitoring 228 vehicles to managing entire multi-modal city networks.

## 1.2 System Summary Table

| Attribute   | Detail                                 |
|-------------|----------------------------------------|
| System Name | AI-Driven PT Operations Decision Agent |
| Platform    | n8n workflow automation + Groq LLM API |
| Language    | JavaScript (n8n toolCode nodes)        |



| Attribute    | Detail                                                                       |
|--------------|------------------------------------------------------------------------------|
| Data Source  | MBTA v3 API (live bus vehicle and trip data)                                 |
| Trigger      | Schedule node, every 2 minutes                                               |
| Sub-Agents   | SA1 Analysis, SA2 Monitoring, SA3 Decisions, SA4 Execution                   |
| Upgrade Path | Agentic AI via LangGraph cycles and self-correction loops (approx. 2 months) |

## 2. System Overview

The AI-Driven Public Transport Operations Decision Agent is an automated, multi-agent AI system designed to monitor, analyze, and optimize public transport operations in real time. The system is built on n8n, a JavaScript-based workflow automation platform, powered by a Groq LLM (llama-3.3-70b-versatile), and integrates with the MBTA v3 API to ingest live bus data every two minutes.

The system implements 12 core functions (Fn1 through Fn12) distributed across four specialized sub-agents (SA1 through SA4), coordinated by a central Main Coordinator Brain. It autonomously detects congestion, assesses delays, classifies operational risk, makes rerouting and frequency decisions, and activates a critical-exception alert only in the rare event that operational confidence drops below the defined threshold.

### 2.1 Stakeholders

| Stakeholder          | Role                     | Interest / Concern                                                 |
|----------------------|--------------------------|--------------------------------------------------------------------|
| Transport Operators  | Primary end users        | Real-time alerts, route decisions, escalation notifications        |
| Dispatchers          | Critical Escalation Only | Receive escalation emails when AI confidence is below 0.6          |
| System Developer     | Builder and maintainer   | n8n workflow integrity, Groq API reliability, JS logic correctness |
| Passengers           | Indirect beneficiaries   | Improved on-time performance, reduced overcrowding                 |
| University Assessors | Evaluators               | Project completeness, technical correctness, documentation quality |

## 3. Requirements Specification

### 3.1 Functional Requirements

Functional requirements define what the system must do. Each requirement is mapped to its corresponding function node and sub-agent.

| ID   | Function | Sub-Agent | Requirement                                                                                                                                                                                                                                           |
|------|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR01 | Fn1      | SA1       | The system shall ingest live bus vehicle data from the MBTA v3 API every 2 minutes, including vehicle position, occupancy status, current status, and route ID.                                                                                       |
| FR02 | Fn2      | SA1       | The system shall clean and normalize all ingested vehicle data, clamping load_factor to [0,1] and delay_minutes to $\geq 0$ , and discard records older than 10 minutes.                                                                              |
| FR03 | Fn3      | SA1       | The system shall detect route-level congestion by grouping vehicles by route and flagging routes where average delay exceeds 5 minutes or average load_factor exceeds 0.85.                                                                           |
| FR04 | Fn4      | SA1       | The system shall assess delays per route, computing average delay and load_factor, and produce a delay_assessment object for downstream agents.                                                                                                       |
| FR05 | Fn5      | SA2       | The system shall monitor vehicle utilization continuously, flagging overloaded vehicles (load > 0.9), underutilized vehicles (load < 0.3), sudden load spikes (> 0.2 in 5 minutes or less), and repeated delays (> 10 min across consecutive cycles). |
| FR06 | Fn6      | SA1       | The system shall classify overall operational risk as Low, Medium, or High based on congested route thresholds (avgDelay > 15 or avgLoad > 0.9 = High; avgDelay > 5 = Medium).                                                                        |
| FR07 | Fn7      | SA3       | The system shall generate rerouting decisions for all High-risk routes, producing a rerouting_decisions array with route_id, alternative_route, reason, and timestamp.                                                                                |
| FR08 | Fn8      | SA3       | The system shall generate frequency adjustment recommendations, increasing frequency to 5-minute intervals for routes with avgLoad > 0.85 and decreasing to 20-minute intervals for routes with avgLoad < 0.3.                                        |
| FR09 | Fn9      | SA4       | The system shall generate maintenance alerts for vehicles with repeated delays across consecutive polling cycles.                                                                                                                                     |
| FR10 | Fn10     | SA4       | The system shall route decisions autonomously to the Action Layer when SA3 confidence is $\geq 0.6$ , and trigger a human escalation email when confidence < 0.6, including full payload details.                                                     |
| FR11 | Fn11     | SA4       | The system shall produce a structured audit log entry for every polling cycle, recording timestamp, risk level, actions taken, and cycle notes.                                                                                                       |
| FR12 | Fn12     | SA4       | The system shall calculate an effectiveness score (0 to 1) for each cycle based on risk level outcome (High = 0.2, Medium = 0.5, Low = 0.8) to measure improvement over time.                                                                         |

### 3.2 Non-Functional Requirements

| ID   | Category        | Metric                 | Requirement                                                                                                                                 |
|------|-----------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| NF01 | Performance     | 30 seconds or less     | The system shall complete a full Fn1 to Fn12 cycle within 120 seconds of the schedule trigger firing.                                       |
| NF02 | Reliability     | 99% or greater uptime  | The n8n workflow shall remain active and polling continuously, with error handling routing failures to the Error Notification Email node.   |
| NF03 | Scalability     | 100+ vehicles          | The system shall handle a minimum of 100 simultaneous vehicle records per polling cycle without degradation.                                |
| NF04 | Accuracy        | 0.6 confidence minimum | SA3 decisions shall only be executed autonomously when confidence score is 0.6 or above; lower confidence always triggers human escalation. |
| NF05 | Security        | API key management     | All API credentials shall be stored as n8n environment variables and never hardcoded in workflow node parameters.                           |
| NF06 | Maintainability | Modular JS nodes       | Each function (Fn1 to Fn12) shall be implemented as a discrete, independently editable JavaScript toolCode node within n8n.                 |
| NF07 | Auditability    | 100% cycle logging     | Every polling cycle shall produce a complete audit log entry regardless of risk level or action taken.                                      |
| NF08 | Usability       | JSON output            | All sub-agent outputs shall be valid JSON with no markdown formatting, ensuring downstream nodes can parse without error handling.          |

### 3.3 Use Cases

#### UC01: Automated Route Optimization Cycle

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor         | System (automated, no human trigger)                                                                                                                                                                                                                                                                                                                                                                                                    |
| Precondition  | n8n workflow is active; Groq API key is configured; MBTA API is reachable.                                                                                                                                                                                                                                                                                                                                                              |
| Postcondition | Rerouting and frequency decisions are executed or escalated; audit log entry is created.                                                                                                                                                                                                                                                                                                                                                |
| Main Flow     | 1. Schedule Trigger fires every 2 minutes. 2. Normalise Input node standardizes the task payload. 3. Main Coordinator Brain dispatches SA1, SA2, SA3, SA4 in parallel. 4. SA1 executes Fn1 through Fn4 and Fn6. 5. SA2 executes Fn5 independently. 6. SA3 executes Fn7 and Fn8. 7. SA4 executes Fn9 through Fn12. 8. Merge1 consolidates all SA outputs. 9. Confidence Switch routes decisions to the Action Layer or escalation email. |

UC02: Human Escalation on Low Confidence

|               |                                                                                                                                                                                                                                                                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor         | Dispatcher (human)                                                                                                                                                                                                                                                                                                                                                          |
| Precondition  | SA3 has produced decisions with confidence below 0.6.                                                                                                                                                                                                                                                                                                                       |
| Postcondition | Dispatcher receives email with full payload; no autonomous actions are executed.                                                                                                                                                                                                                                                                                            |
| Main Flow     | 1. Confidence Switch routes output to Format Escalation node. 2. Escalation email is composed with risk level, affected routes, anomalies, and pending decisions. 3. Email is sent to the ESCALATION_EMAIL environment variable address. 4. Dispatcher reviews the payload and takes manual action if required. 5. Next polling cycle begins independently after 2 minutes. |

UC03: High-Risk Congestion Response

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor         | System (autonomous)                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Precondition  | SA1 has classified risk as High (avgDelay > 15 or avgLoad > 0.9 on any route).                                                                                                                                                                                                                                                                                                                                                                     |
| Postcondition | Rerouting decisions and frequency increases are logged and executed; maintenance alerts raised.                                                                                                                                                                                                                                                                                                                                                    |
| Main Flow     | 1. SA1 Fn6 classifies risk as High and outputs affected route IDs. 2. SA3 Fn7 generates rerouting decisions for all High-risk routes. 3. SA3 Fn8 recommends 5-minute frequency intervals for overloaded routes. 4. SA4 Fn9 flags any vehicles with repeated delays for maintenance. 5. Action Layer executes all decisions and logs results with timestamps. 6. Fn12 scores cycle effectiveness at 0.2 (High risk equals a low improvement score). |

3.4 Constraints and Assumptions

Constraints

| ID  | Category             | Description                                                                                                                                                        |
|-----|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C01 | Platform             | The system must run within the n8n workflow environment. No standalone Python or other runtime environments are used for core logic.                               |
| C02 | LLM Provider         | The system depends on Groq API availability. A Groq API key must be configured as an n8n credential before activation.                                             |
| C03 | Data Source          | The system is designed for the MBTA v3 API (Boston bus network). Adapting to another transit agency requires modifying Fn1 field mappings and occupancy maps.      |
| C04 | Polling Interval     | The minimum reliable polling interval is 2 minutes, constrained by MBTA API rate limits and n8n schedule trigger resolution.                                       |
| C05 | Confidence Threshold | The autonomous action threshold of 0.6 is fixed in the SA3 system prompt. Adjusting it requires editing the Confidence Switch node condition in n8n.               |
| C06 | Agentic Upgrade      | LangGraph-style self-correction cycles and dynamic SA spawning are out of scope for v1.0, requiring approximately 2 additional months of development.              |
| C07 | Budget               | Development cost is estimated at USD \$68,960 to \$172,400 depending on team size and hourly rate, based on a COCOMO estimate of approximately 8.62 person-months. |

## 4. Design Techniques

The system design is documented through a series of diagrams covering different architectural perspectives. Each diagram addresses a specific concern, from the high-level system boundary to the internal flow of user authentication and agent communications.

### 4.1 Context Diagram

The context diagram provides a bird's eye view of the entire system. It shows the AI system as a single hub and illustrates how data inputs such as GPS and weather flow into the system, while instructions such as reroutes and alerts flow out to external actors including the fleet and dispatchers. This diagram defines the boundaries: what is part of the software and what is an external dependency.

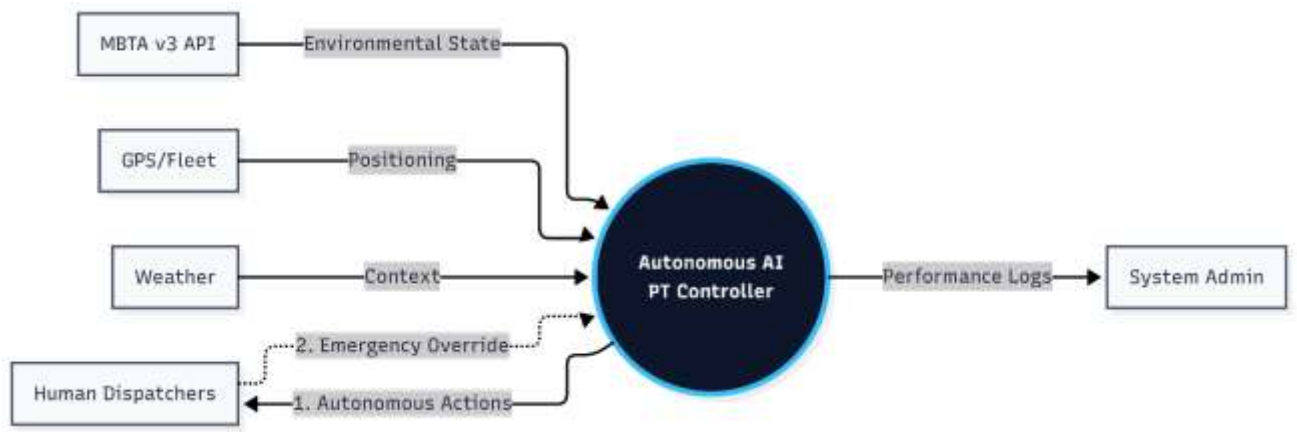


Figure 4.1: System Context Diagram

### 4.2 Layered Architecture Pattern Diagram

The layered architecture organizes the codebase into a stack. Data starts at the Input Layer, is processed by the Reasoning Layer (the Brain), managed by the Sub-Agents, and finally executed in the Output Layer. This ensures separation of concerns, making the system easier to debug and scale.

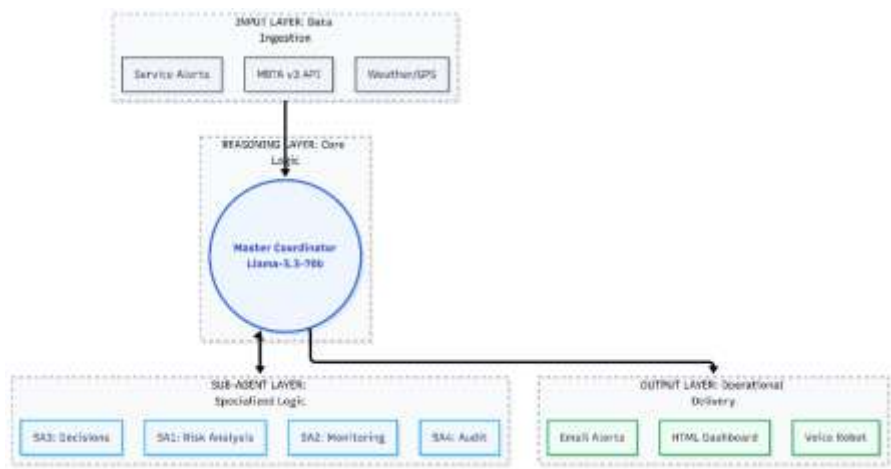


Figure 4.2: Layered Architecture Pattern Diagram

### 4.3 Activity Diagram: Access Attempt

This diagram tracks the step-by-step process of a user logging into the system: entering a user ID, the system checking the database, and the branching path to either the dashboard (success) or an error message (failure). It visualizes the if-then logic governing user security.

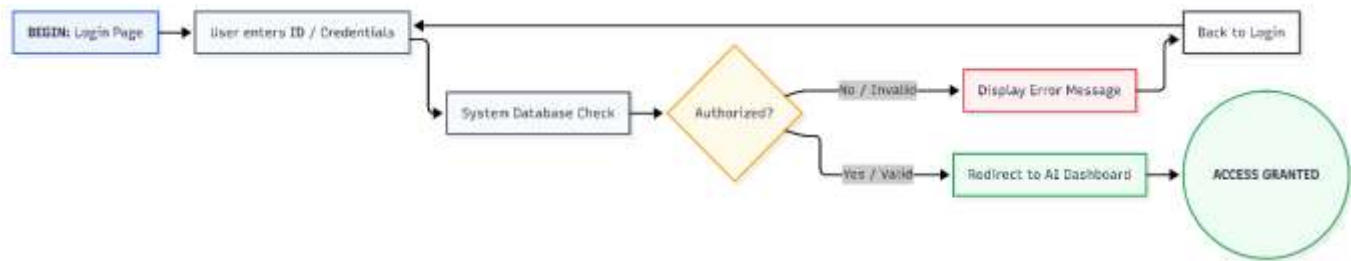


Figure 4.3: Activity Diagram for User Access Attempt

### 4.4 Model-View-Controller Diagram

The MVC diagram shows how the system components interact internally. The View (Dashboard) sends user requests to the Controller (AI Engine), which updates the Model (Transport Data), which then refreshes the View. This keeps the user interface independent from the complex AI logic underneath.

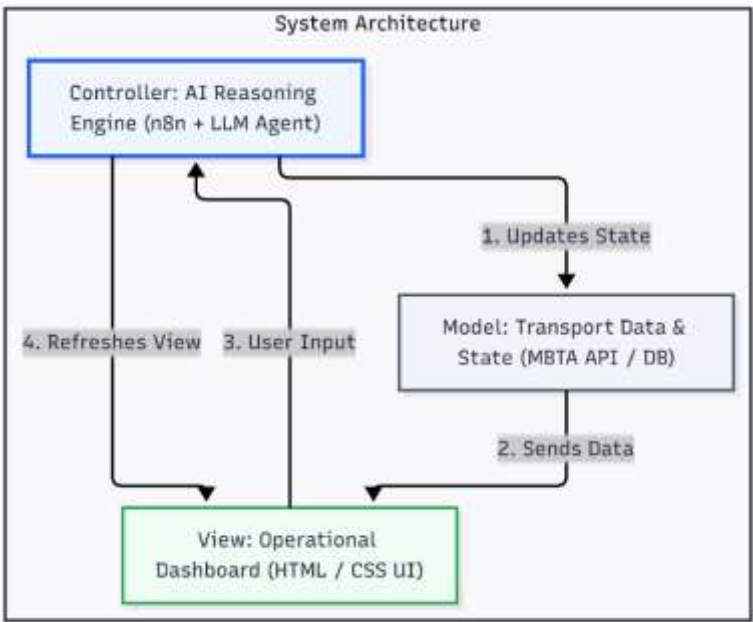


Figure 4.4: Model-View-Controller Diagram

### 4.5 Use Case Diagram

This diagram maps specific actions to the actors responsible for them, providing a functional requirement summary for stakeholders.

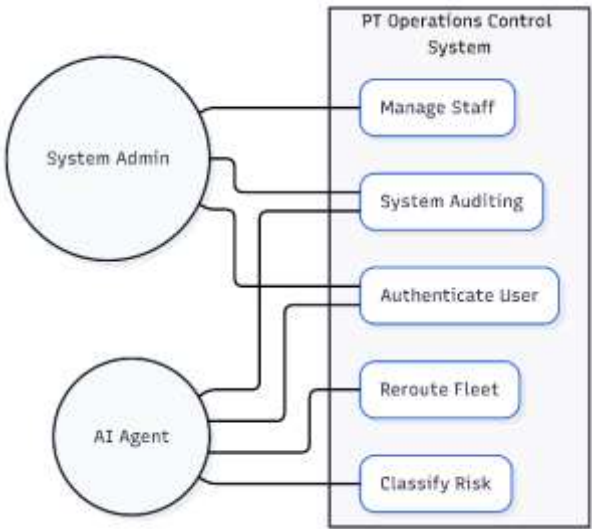


Figure 4.5: Use Case Diagram

### 4.6 Use Case Tables

| UC ID | Name              | Primary Actor | Description                                                                        |
|-------|-------------------|---------------|------------------------------------------------------------------------------------|
| UC-01 | Authenticate User | Dispatcher    | Validates User ID against the database to grant dashboard access.                  |
| UC-02 | Classify Risk     | AI Sub-Agent  | Analyzes GPS and delay data to categorize operational risk (Low, Medium, or High). |
| UC-03 | Reroute Fleet     | AI Sub-Agent  | Autonomously calculates and pushes new route data to vehicles via n8n.             |
| UC-04 | System Auditing   | Admin         | Reviews the decision log for performance evaluation and testing.                   |
| UC-05 | Manage Staff      | Admin         | Allows the addition or deletion of authorized user IDs from the system.            |

### 4.7 Sequence Diagram: Adding and Deleting a User

This diagram shows the timeline of a specific task, tracking the chronological messages sent between the Admin, the System, and the Database to successfully update user permissions. It highlights the order of operations and where potential delays or errors such as a database timeout might occur.

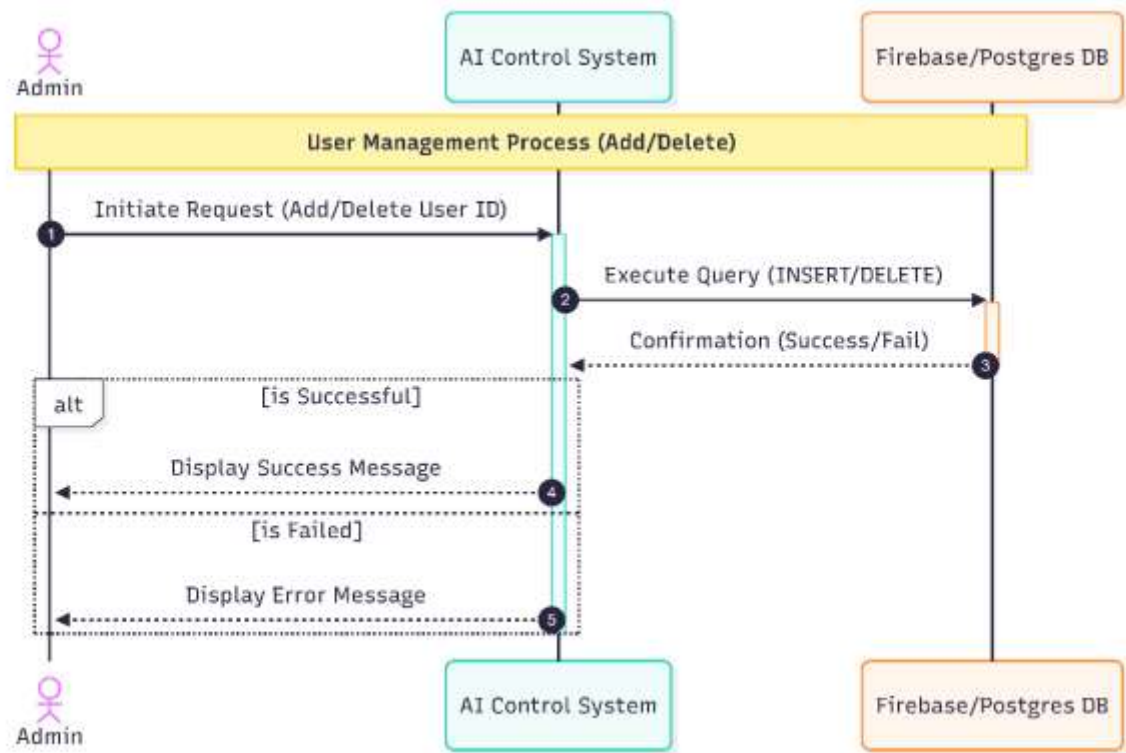


Figure 4.7: Sequence Diagram for Adding or Deleting a User



### 4.8 Activity Diagram: System Communications

This diagram tracks the automated lifecycle of the agent. It shows the system constantly polling data, running it through the 12 core functions, and deciding whether to trigger an external action via n8n or simply log the status and wait. This is the engine room view of the AI's autonomy.

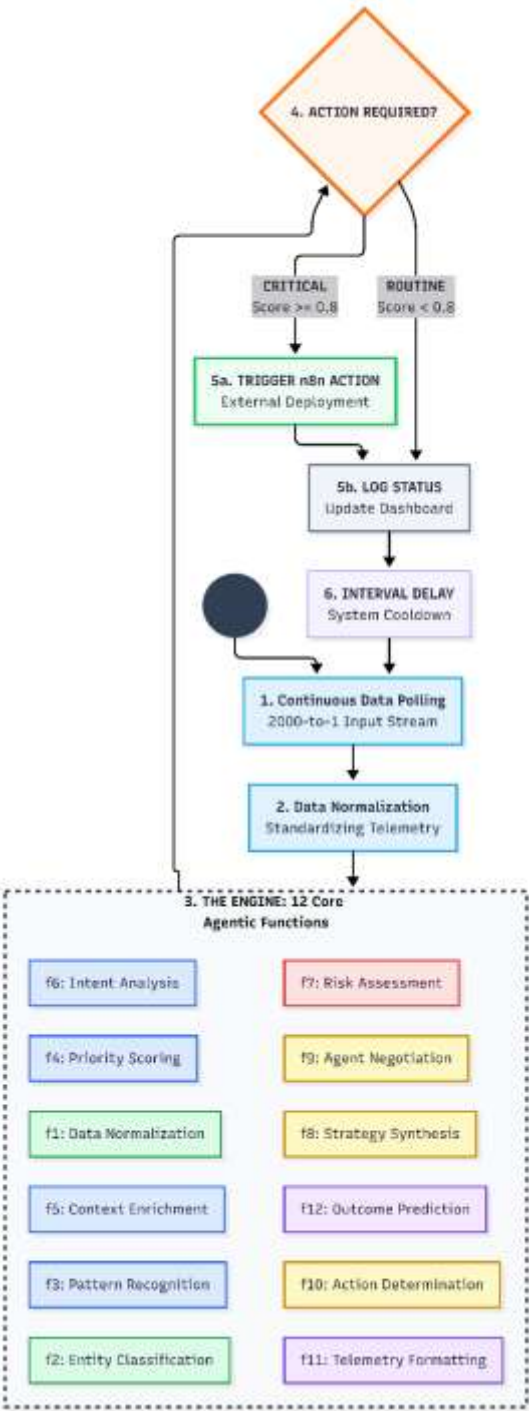


Figure 4.8: Activity Diagram for System Communications

### 4.9 Sequence Diagram: User Request

This diagram focuses on the end-to-end execution of an optimization request. A user asks for an optimization; the AI processes it and communicates with the n8n hub to send out real-world notifications via email or messaging services. It demonstrates how the AI agent bridges the gap between a user request and a real-world action.

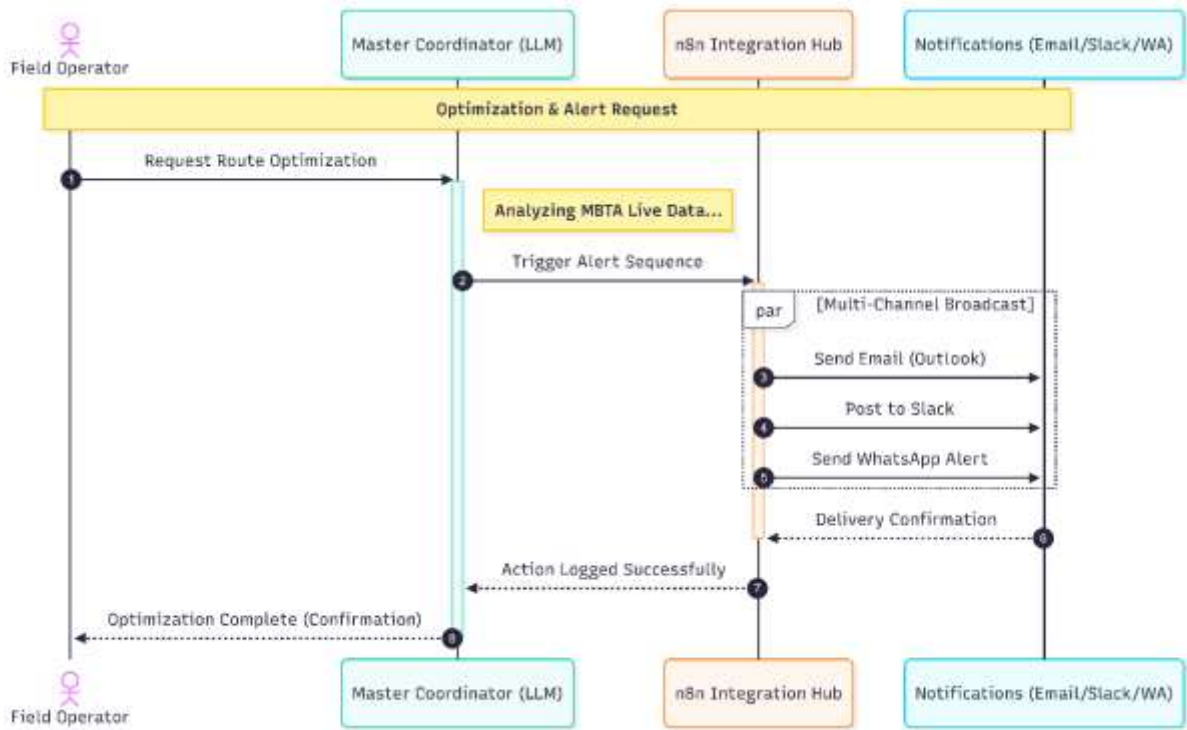


Figure 4.9: Sequence Diagram for User Request Execution

## 5. Implementation

This project implements a multi-agent AI system for public transport operations optimization using n8n, LangChain agents, and real-time transport data APIs. The system continuously monitors transport conditions, detects congestion and anomalies, assesses operational risk, and executes routing and scheduling decisions entirely on its own. It operates as a fully autonomous decision-making pipeline. Human operators are contacted only in the rare event that the system's confidence score drops below its operational threshold.

### 5.1 System Architecture Overview

The pipeline follows this flow: Schedule Trigger initiates Data Ingestion, which feeds into Data Processing, which feeds into the AI Multi-Agent Layer, which feeds into the Decision Layer, and finally into the Output Layer. The key components are the Trigger Layer, Data Pipeline, AI Multi-Agent Layer (Coordinator and SA1 through SA4), Decision and Action Layer, and Output and Notification Layer.

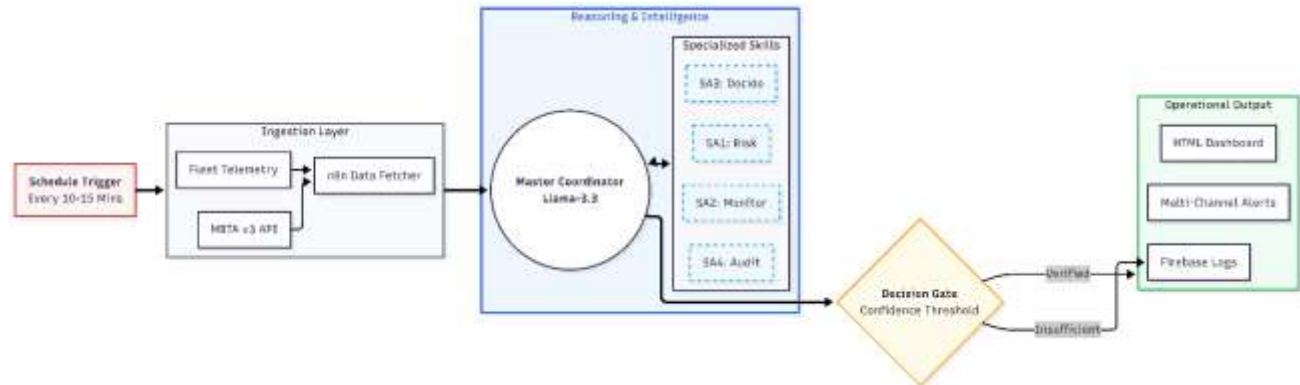


Figure 5.1: System Architecture Overview

## 5.2 Trigger and Input Handling

The system uses an n8n Schedule Trigger configured to run every two minutes, ensuring continuous monitoring. An Input Normalization node then standardizes the incoming task payload, setting a default task of "analyze current transport conditions" and a default output mode of "dashboard" if none are provided. This ensures a consistent data structure for all downstream agents.

## 5.3 Data Ingestion Layer

The transport API integration calls the MBTA v3 API vehicles endpoint using an HTTP request in JavaScript. The ingestion function captures vehicle speed, status, occupancy, and location data. The output includes a total vehicle count and a sample array for downstream processing.

## 5.4 Data Processing Layer

### 5.4.1 Congestion Detection (Fn3 and Fn4)

Vehicles are grouped by route, and two metrics are computed per route: average delay in minutes and average load factor. A route is flagged as congested if its average delay exceeds five minutes or its average load factor exceeds 0.85.

### 5.4.2 Risk Classification (Fn6)

Risk levels are assigned based on computed route metrics. A route with an average delay above 15 minutes or an average load above 0.9 is classified as High risk. Routes with average delays above five minutes but below the High threshold are classified as Medium. All other routes are classified as Low risk.

### 5.4.3 Anomaly Detection (Fn5)

The anomaly detection function flags overloaded vehicles (load factor above 0.9), repeated delays across consecutive cycles, and sudden load spikes greater than 0.2 within a five-minute window.

### 5.4.4 Frequency Adjustment (Fn8)

When a route's average load exceeds 0.85, the system recommends increasing service frequency to five-minute intervals. When a route's average load falls below 0.3, the system recommends decreasing frequency to 20-minute intervals to avoid wasting resources.

## 5.5 Multi-Agent AI Layer

### 5.5.1 Main Coordinator Agent

The Main Coordinator controls the overall system workflow and outputs a structured task JSON that specifies the task type and desired output mode. It operates at a low temperature (0.2) to ensure deterministic, consistent dispatching to the four sub-agents.

### 5.5.2 Specialized Sub-Agents

SA1, the Risk Analysis Agent, evaluates congestion and outputs risk level, delay insights, and key routes. SA2, the Anomaly Detection Agent, identifies system anomalies and utilization issues and decides which agent to route to next. SA3, the Optimization Agent, generates rerouting decisions and frequency adjustment recommendations. SA4, the Execution and QA Agent, produces maintenance alerts, audit logs, and escalation signals, and scores each cycle's effectiveness.

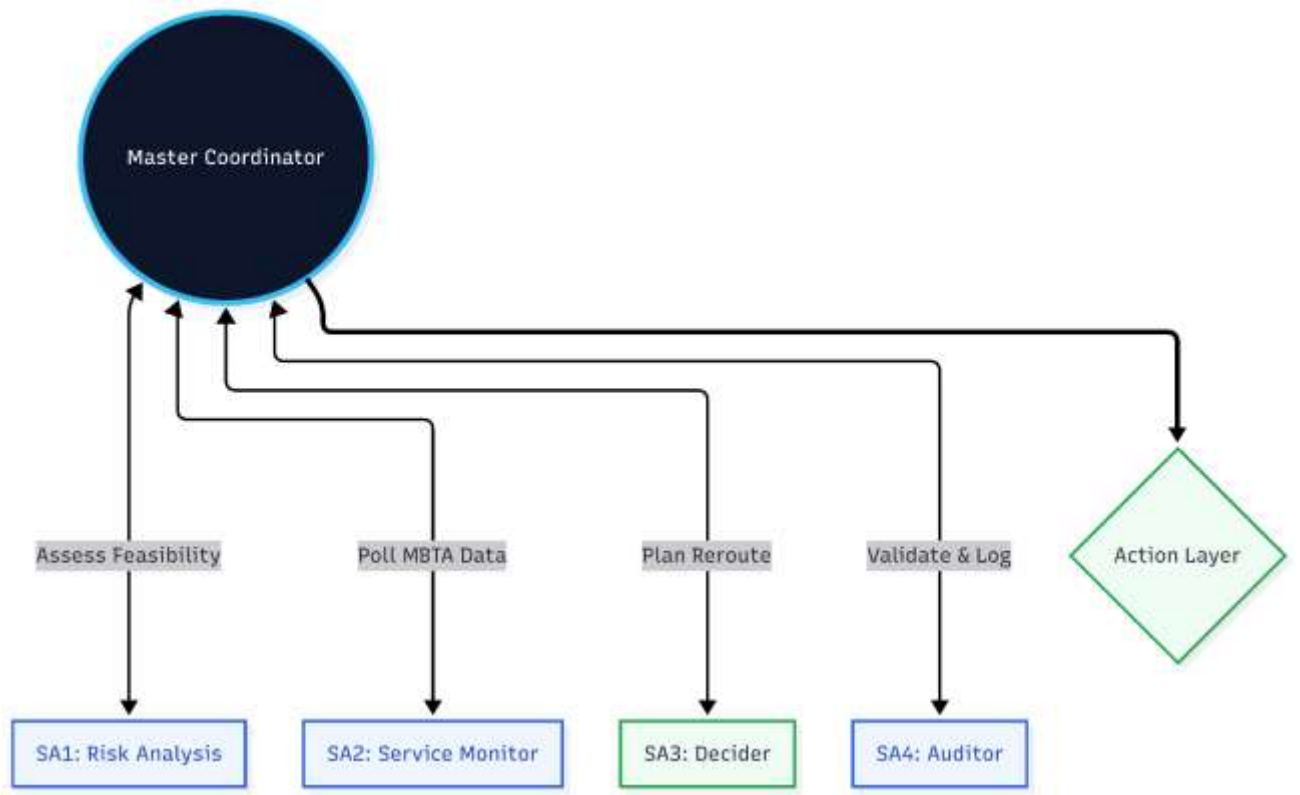


Figure 5.5: Multi-Agent Hub-and-Spoke Architecture

### 5.5.3 Decision and Aggregation Layer

After all four agents complete their cycles, the Merge1 node collects their outputs and a consolidation code node walks through them, picking out the fields it needs. It handles both raw JSON and stringified JSON because LLM agents sometimes wrap their output in a string. The result is a unified object containing risk level, confidence score, congested routes, rerouting decisions, anomalies, and frequency adjustments.

### 5.5.4 Action Layer

The Action Layer converts SA3's decisions into a structured action log. Each reroute or frequency change receives a timestamp and status so SA4 can audit them and Fn12 can score the cycle's effectiveness. If nothing requires action, the layer still logs a no\_action entry to ensure a complete audit trail.

## 5.6 Traffic Index Calculation

The Traffic Index is calculated using the formula:  $\text{Index} = (\text{avgDelay multiplied by } 0.5) + (\text{avgLoad multiplied by } 5)$ , producing a scale from 0 to 10. A score below 4 indicates Green (normal operations), between 4 and 7 indicates Yellow (moderate concern), and above 7 indicates Red (critical congestion).

## 5.7 Output Layer

The Output Mode Switch routes results to one of three destinations: a full HTML dashboard report, a plain text summary, or a voice output. Email reporting generates a complete HTML dashboard that includes fleet statistics, active alerts, route performance metrics, and the traffic index. The escalation system triggers when AI confidence falls below 0.6, immediately sending a formatted email to the configured operator address.

## 5.8 Memory Management

The system uses n8n's memoryBufferWindow to maintain short-term context across consecutive runs. This provides trend awareness and allows agents to compare current readings against recent history. Vehicle history is persisted in workflow static data across polling cycles, tracking load\_factor, delay\_minutes, and timestamps per vehicle. Session memory keys for the coordinator and sub-agents are configured with context windows of 20, 10, and 30 cycles respectively.

## 5.9 Simplified Simulation Flow

To validate the core decision logic in isolation before connecting live data, a simplified simulation mode was built using a manual trigger and mock data. This allows testing of the SA1 decision logic without requiring access to the MBTA API or the full Groq pipeline.

The simulation flow works as follows. A manual trigger starts the process, representing the point where an operator initiates a test scenario. Fn1 in simulation mode acts as a data generator, injecting mock vehicle data with configurable speed and load values. Fn2 then acts as a pre-processor, stripping unnecessary fields and computing the Delay Score that the decision logic requires. SA1 applies its core if-then logic: if vehicle speed indicates a traffic jam (below 10 mph), the flow routes to the Reroute outcome; if speed is normal, the system logs a No Change outcome and waits for the next cycle. This simplified loop was used extensively during early development to confirm that the threshold and branching logic was working correctly before the full agent stack was connected.

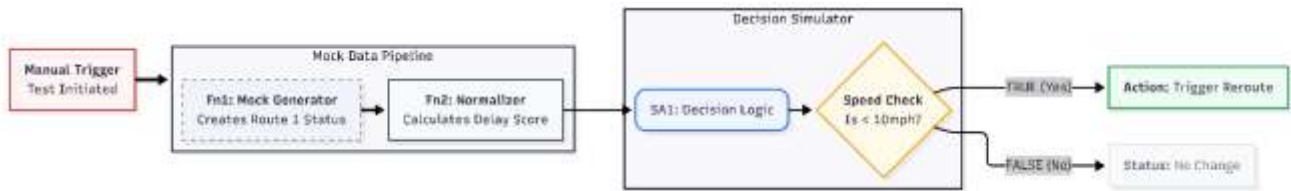


Figure 5.9: Simplified AI Agent Simulation Flow

5.10 Multi-Agent Orchestration Workflow

The full orchestration workflow connects all components from the schedule trigger through to final output delivery. The workflow begins with Ingestion and Normalization (Fn1 and Fn2). The Schedule Trigger fires every two minutes, pulling live data from the MBTA API and passing it through the normalization node to structure it for the LLM agents. The Main Coordinator then receives the cleaned data and manages the overall conversation state, delegating tasks to the four sub-agents.

The Strategic Layer processes data in parallel. SA1 analyzes risk based on traffic delays and route loading. SA2 monitors for sudden anomalies such as a bus stopped far longer than expected at a location. SA3 generates the optimized decisions including ethical fairness checks to ensure lower-density routes are not neglected in favor of high-traffic corridors. SA4 audits the entire decision chain and assigns a Confidence Score to the final recommendation.

The Safety Switch is the final step before any action is taken. If the AI confidence drops below 60%, the system treats this as a critical exception. It suspends the current cycle and forwards the full decision payload to a human dispatcher for review. This path is an exceptional safety circuit, not a routine operational mode. If confidence meets the threshold, successful decisions are pushed to the Firebase HTML Dashboard and can also be delivered via the voice output mode.



Figure 5.10: Multi-Agent Orchestration Workflow

5.11 Fully Agentic AI Workflow

The Version 1.0 system operates on a fixed scheduled trigger, executing the same 12-function pipeline every two minutes. The fully agentic upgrade replaces this rigid structure with a dynamic reasoning model where the Master Coordinator decides which tools to invoke based on what the live data is showing, rather than following a predetermined sequence.

### 5.11.1 Key Architectural Upgrades

Autonomous Reasoning replaces the linear step-by-step flow. The Master Coordinator evaluates incoming data and selects which tool to call next based on the risk profile it detects. This represents the shift from a multi-agent system with fixed roles to a truly agentic system with dynamic roles that adapt to each operational cycle.

Token Management is handled through strategically placed Wait nodes inserted before high-compute tasks such as the Risk Analysis Agent and final report generation. This prevents the system from hitting API rate limits during high-density data cycles when multiple routes are flagged simultaneously.

Tool-Based Architecture reclassifies the MBTA interactions as callable tools rather than fixed pipeline steps. The agent can call the `verify_live_vehicles` tool multiple times within a single cycle if it needs additional data before committing to a routing decision, adding a layer of self-verification not possible in the fixed pipeline.

The Escalation Router serves as the final filter in the agentic workflow, determining whether an autonomous decision is safe to execute directly or requires immediate human flagging. This router replaces the simpler binary confidence switch with a more nuanced evaluation that considers both confidence score and the nature of the proposed action.

### 5.11.2 Agentic Upgrade Path

The upgrade from the current Version 1.0 architecture to a fully agentic system involves three main changes. First, the fixed toolCode node chains are replaced with ReAct-loop agents that self-select which tools to call based on intermediate reasoning steps. Second, LangGraph is added to enable stateful cycles, allowing agents to loop back and gather more data before reaching a decision rather than always progressing forward. Third, a self-correction mechanism is introduced so that if Fn2 returns fewer vehicles than expected due to data quality issues, SA1 automatically retries ingestion with adjusted filter parameters rather than proceeding with incomplete data.

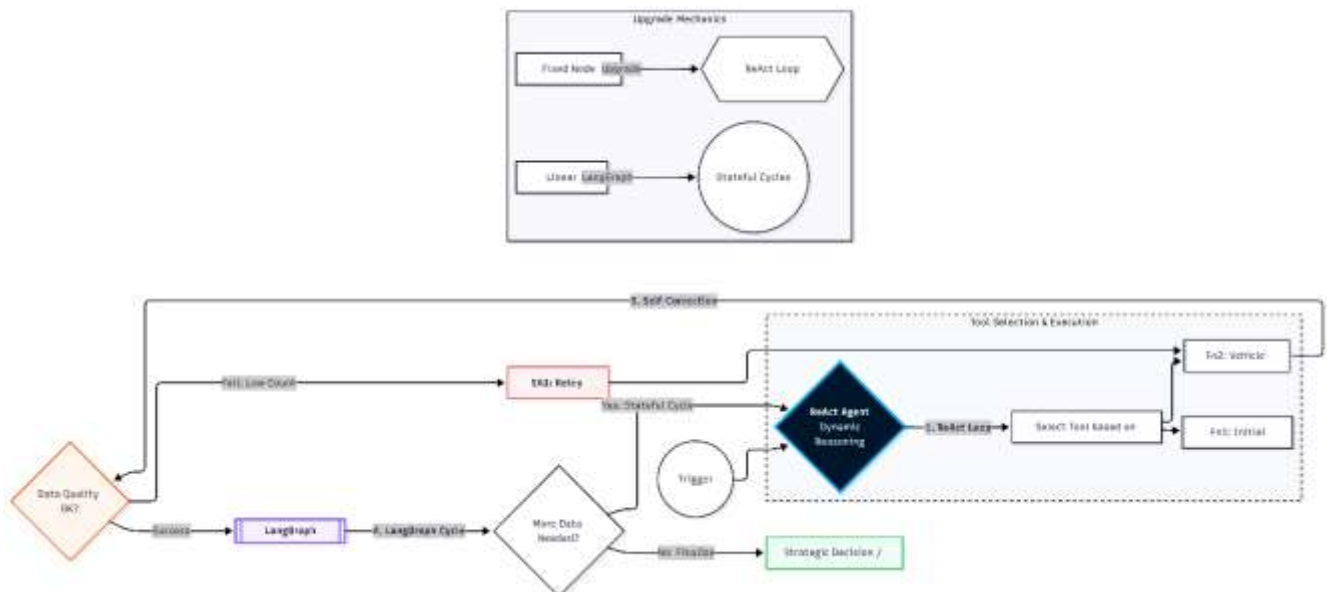


Figure 5.11: Fully Agentic AI Workflow Architecture



## 5.12 Implementation Strengths and Improvement Areas

The current implementation demonstrates several notable strengths. The modular design makes individual functions easy to debug and extend. The hybrid approach combining AI reasoning with rule-based thresholds provides predictable behavior at the edges while allowing intelligent decision-making in ambiguous cases. Real-time adaptability through the two-minute cycle keeps decisions grounded in current conditions. The overall workflow structure is production-ready, with proper error handling, audit logging, and escalation pathways in place.

Areas identified for future improvement include adding machine learning prediction models for delay forecasting so the system can anticipate congestion rather than only react to it, integrating historical data storage to support longer-term trend analysis, adding geospatial visualization for map-based operator views, applying reinforcement learning to optimization decisions so the system improves routing choices based on real-world outcomes, and upgrading the anomaly detection module to use time-series models that can better distinguish genuine anomalies from temporary sensor fluctuations.

## 6. Build and Demonstration Record

---

This section documents the progressive development journey undertaken across three distinct phases of AI agent research and implementation. Each phase was independently researched, designed, built, and demonstrated as a functional working system. The work progressed from a foundational single-agent system through a coordinated multi-agent architecture and culminated in a fully agentic workflow capable of dynamic reasoning and self-correction. Alongside these agent builds, a mobile risk alert application, an analytics dashboard website, and a startup concept were developed to demonstrate the practical application of the underlying AI systems.

### 6.1 Basic AI Agent

#### 6.1.1 Research

The research phase for the basic AI agent involved a systematic review of how large language models could be embedded within workflow automation platforms to act as autonomous decision nodes. Key areas studied included LLM agent design patterns, the capabilities and rate limits of the Groq API versus competing providers, and the toolCode node architecture within n8n that enables JavaScript functions to be called as agent tools. The MBTA v3 API was evaluated as the live data source, and its vehicle endpoint schema was mapped to the input fields required for basic congestion assessment.

#### 6.1.2 Build

The basic AI agent was built as a single-agent n8n workflow. A Schedule Trigger fires every two minutes and calls the MBTA API to retrieve live vehicle data. A single Groq LLM agent node receives the cleaned JSON payload, applies a straightforward risk classification based on average delay and load metrics across Fn1 through Fn6, and outputs a structured risk assessment object. The HTML dashboard output node renders a one-page status report showing fleet size, average delay, risk level, and a plain-language explanation of the decision.



6.1.3 Demonstration

The basic agent was demonstrated as a live running workflow in n8n. The demonstration showed the Schedule Trigger firing on its two-minute cycle, the LLM agent processing a live MBTA data payload, and the resulting HTML report being generated with a Low, Medium, or High risk classification. The n8n execution log was shown alongside the output to demonstrate the complete thought-to-action chain from raw API data to structured decision.

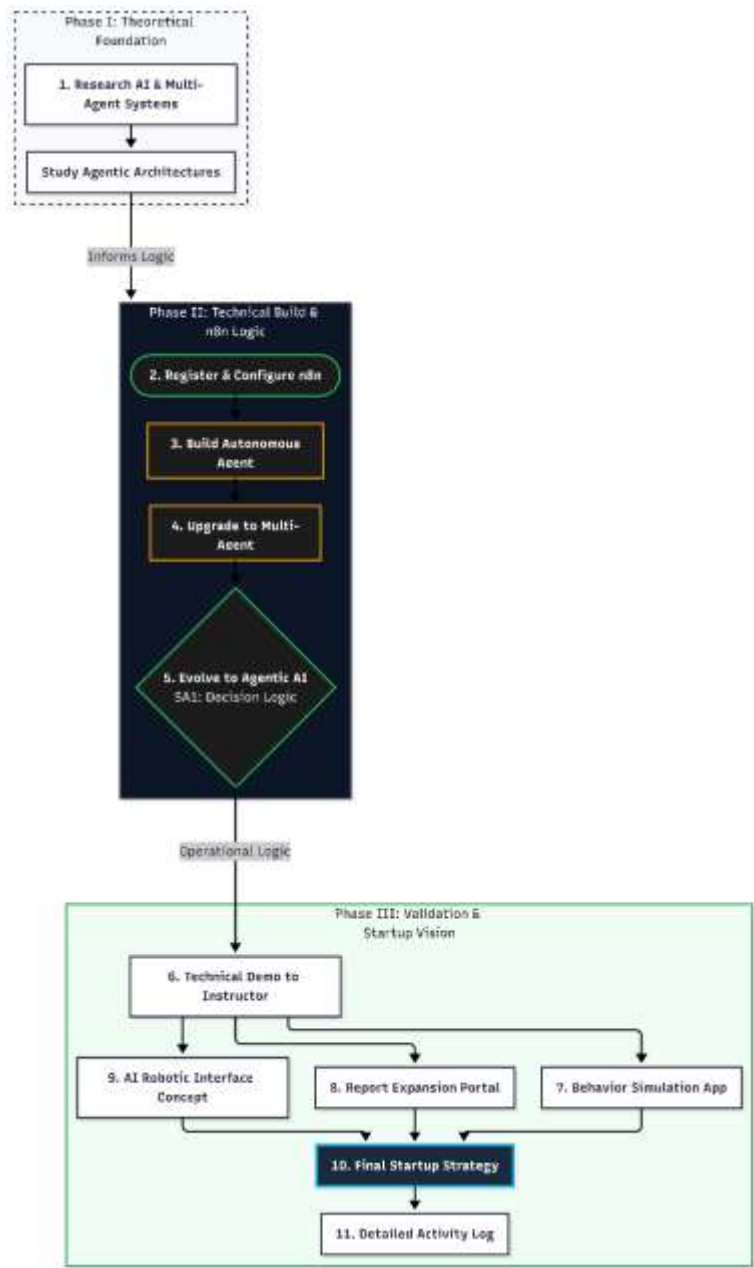


Figure 6.1: Basic AI Agent Research, Build and Demonstration Flow

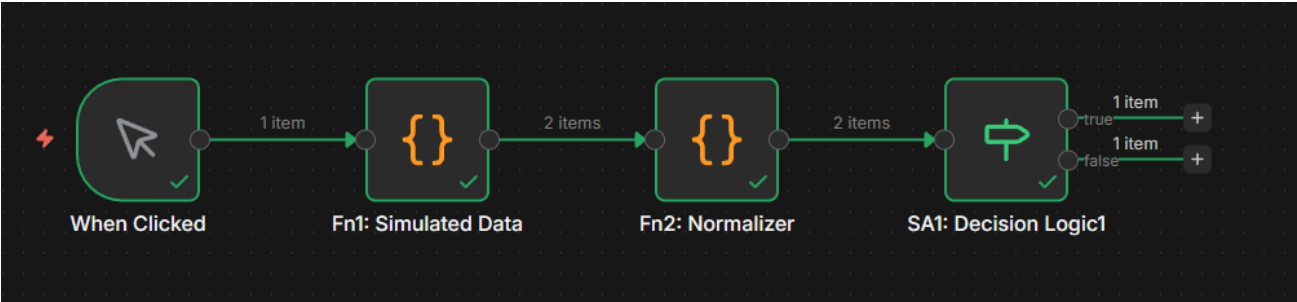


Figure 6.1a: Basic AI Agent — n8n Execution Screenshot

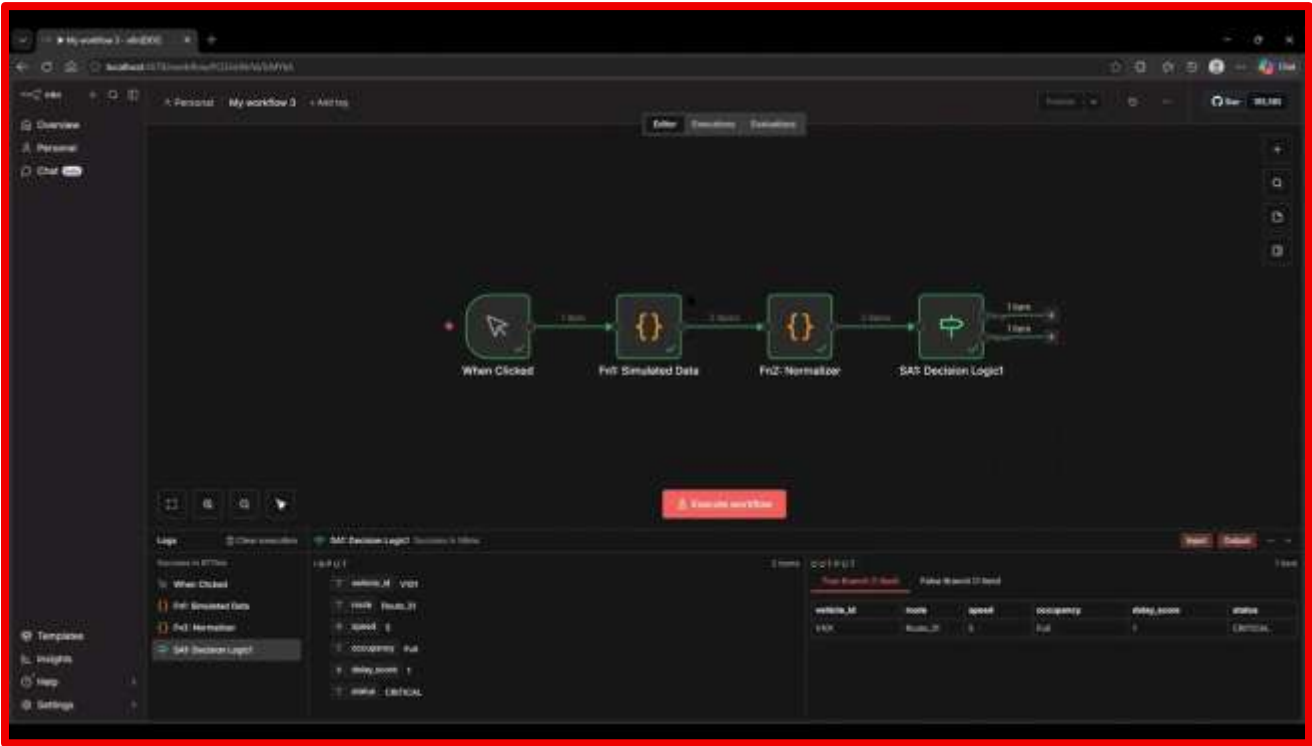


Figure 6.1b: Basic AI Agent — n8n Execution Demo

## 6.2 AI Multi-Agent System

### 6.2.1 Research

Building on the single-agent foundation, the multi-agent research phase examined how complex cognitive tasks could be distributed across specialized AI roles. Academic literature on multi-agent systems (MAS) was reviewed alongside practical LangChain agent documentation to understand how a hub-and-spoke architecture could be implemented within n8n. The four strategic agent roles were defined during this phase: SA1 as the Risk Analyst, SA2 as the Anomaly Detector, SA3 as the Decision Maker, and SA4 as the Compliance Auditor. Confidence scoring methodology and the 60% escalation threshold logic were

established through research into autonomous decision-making safety requirements for public infrastructure.

6.2.2 Build

The multi-agent system was built as an extension of the basic agent, expanding the single LLM node into four parallel LangChain agent nodes coordinated by a Main Coordinator Brain. The full 12-function pipeline (Fn1 through Fn12) was implemented across the four sub-agents. SA1 handles ingestion, cleaning, congestion detection, and risk classification. SA2 runs anomaly detection independently in parallel. SA3 generates rerouting decisions and frequency adjustments. SA4 manages maintenance alerts, audit logging, and the confidence gate that determines whether decisions are executed autonomously or escalated. The Merge1 node consolidates all four agent outputs into a unified decision object.

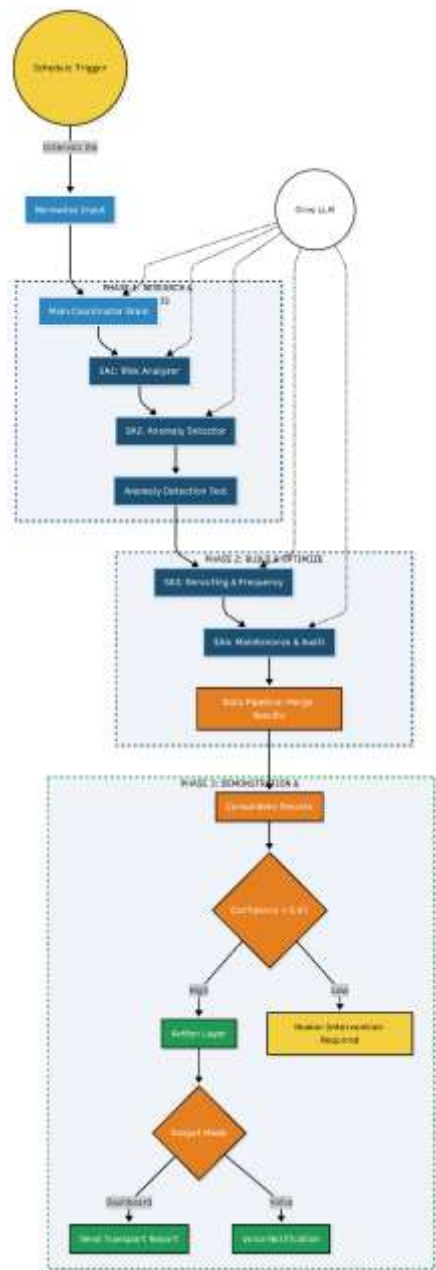


Figure 6.2: AI Multi-Agent System Research, Build and Demonstration Flow

### 6.2.3 Demonstration

The multi-agent demonstration showed the full SA1 through SA4 chain executing in sequence within a single two-minute cycle. A live MBTA data pull was shown being processed through each sub-agent, with the Merge1 node combining outputs and the Confidence Switch routing the final decision to the autonomous dashboard path. A High-risk scenario was also demonstrated by injecting elevated delay values, triggering the rerouting decision and showing the HTML dashboard report with the active alert banner.

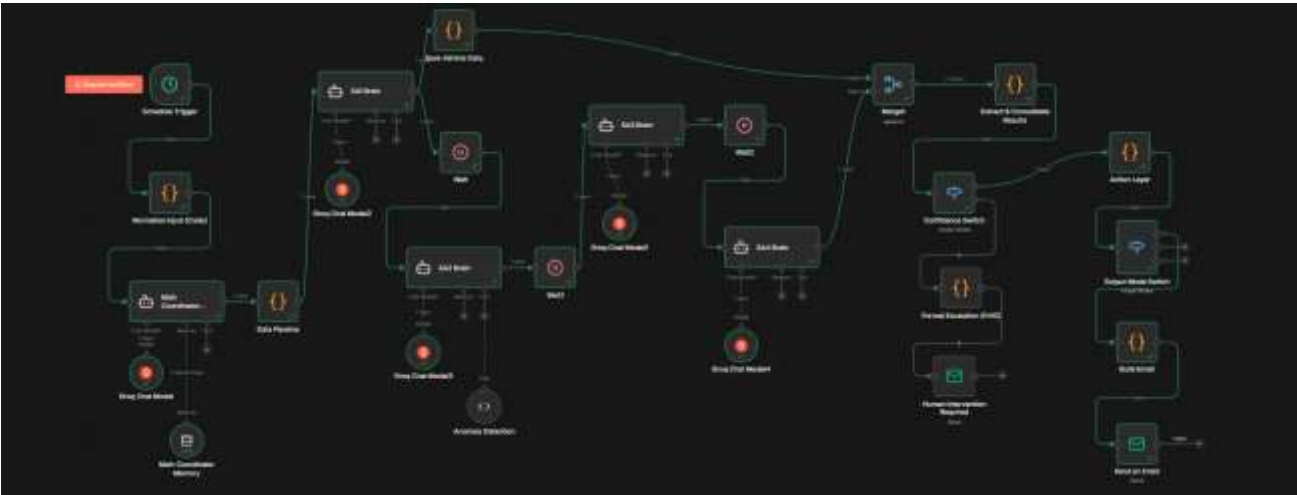


Figure 6.2a: AI Multi-Agent System — n8n Execution Screenshot

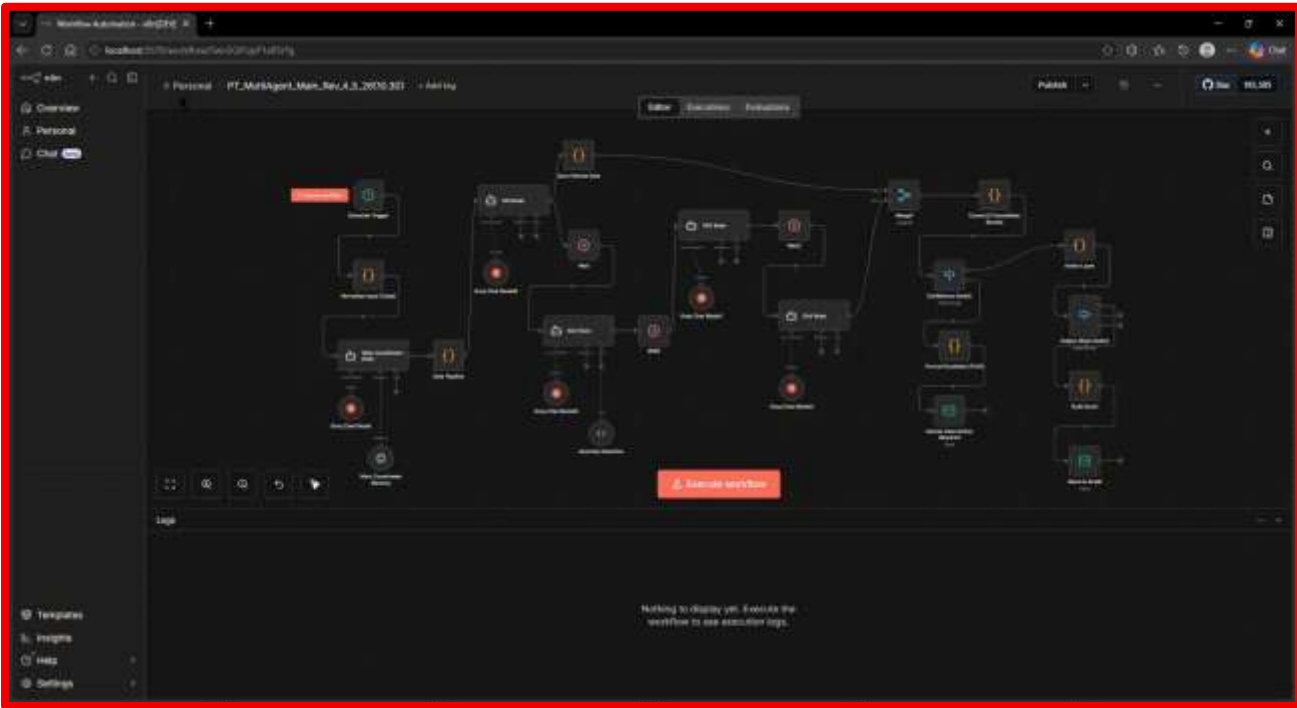


Figure 6.2b: AI Multi-Agent System — n8n Execution Demo

## 6.3 Fully Agentic AI Workflow

### 6.3.1 Research

The agentic research phase examined the architectural shift from fixed-role multi-agent systems to truly autonomous reasoning agents. The ReAct (Reason + Act) loop pattern was studied in depth, along with LangGraph stateful cycle implementations and dynamic tool selection strategies. Token management patterns for high-throughput LLM workflows were investigated, leading to the Wait node placement strategy used to prevent Groq API rate limit breaches during dense data cycles. Research also covered self-correction mechanisms, specifically how an agent can detect poor-quality input and retry data ingestion with adjusted parameters rather than propagating bad data through the pipeline.

### 6.3.2 Build

The fully agentic workflow replaced the fixed tool-chain architecture with a Master Coordinator capable of dynamic tool selection. Rather than executing Fn1 through Fn12 in a predetermined sequence, the Master Coordinator evaluates incoming data and selects which tools to invoke based on the current risk profile. The `verify_live_vehicles` tool can be called multiple times within a single cycle if initial data quality is poor. Wait nodes were strategically placed before the Risk Analysis Agent and before final report generation to manage token throughput. The Escalation Router at the end of the cycle applies a nuanced evaluation of both confidence score and action severity before deciding whether to execute autonomously or flag for human review.

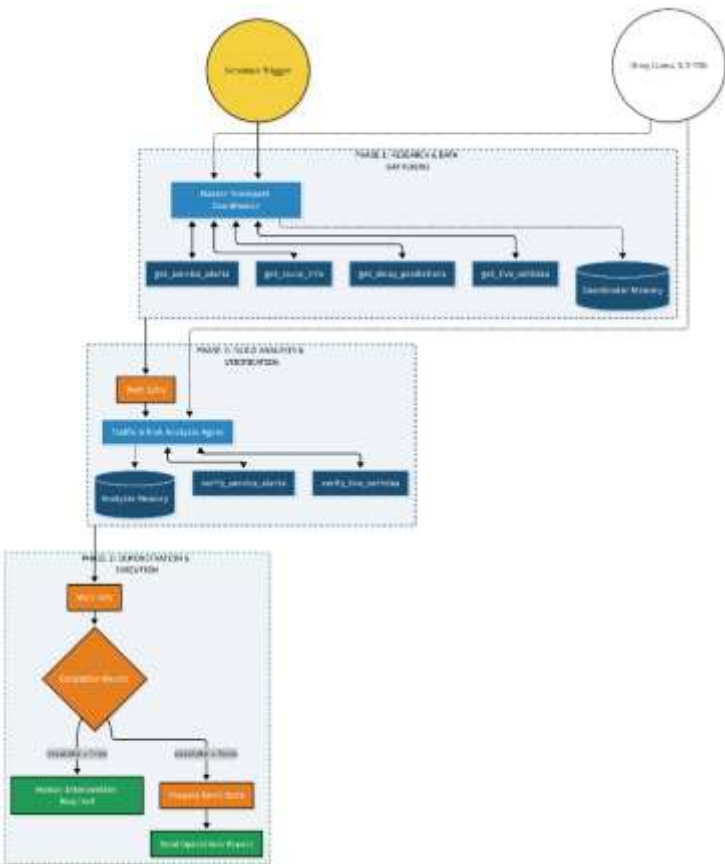


Figure 6.3: Fully Agentic AI Research, Build and Demonstration Flow

### 6.3.3 Demonstration

The agentic demonstration showed the Master Coordinator selecting tools dynamically across two live cycles. In the first cycle, normal conditions were present and the agent completed the cycle in under 45 seconds using a minimal tool call sequence. In the second cycle, a data quality issue was deliberately introduced; the agent was shown detecting the anomaly, invoking the retry loop, recovering clean data, and completing its decision cycle without any manual intervention. The Escalation Router path was also demonstrated, showing how a borderline confidence score triggers a more detailed secondary evaluation before a final routing decision is made.



Figure 6.3a: Fully Agentic AI Workflow — n8n Execution Screenshot

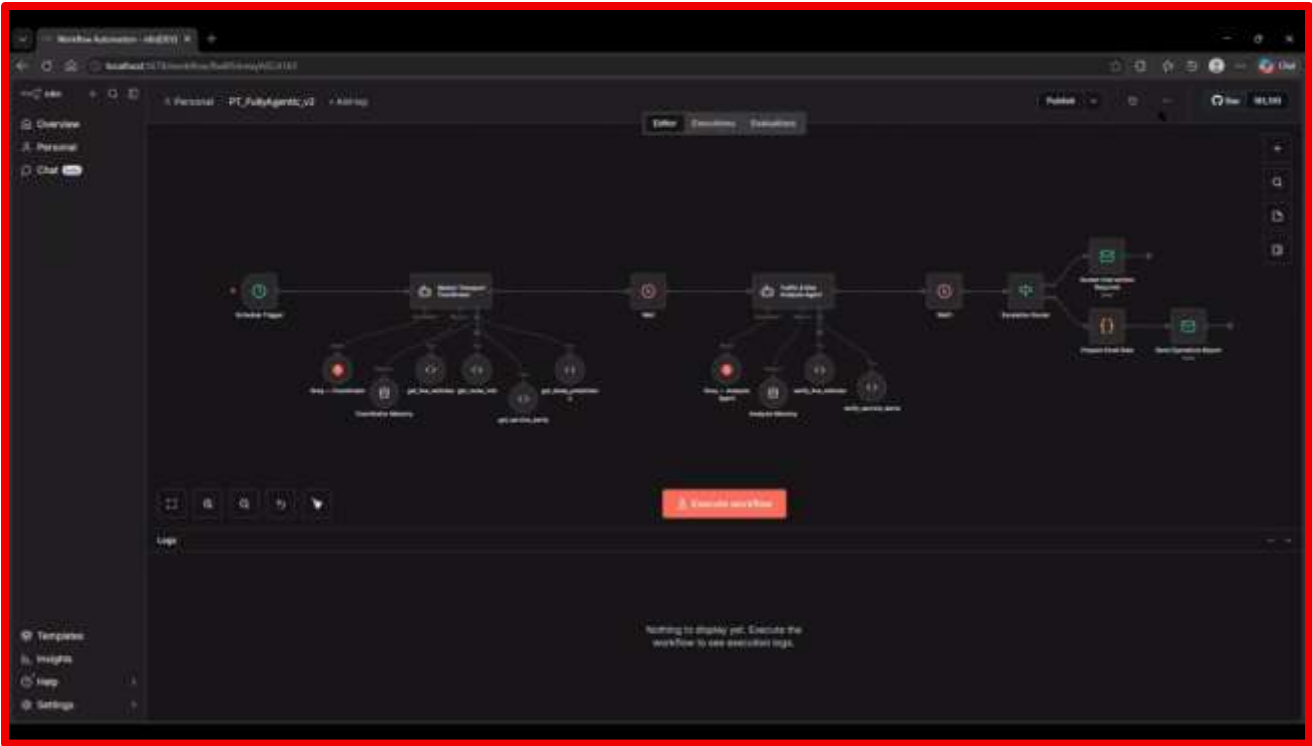


Figure 6.3b: Fully Agentic AI Workflow — n8n Execution Demo

### 6.4 Mobile Risk Alert Application

A mobile application was developed using MIT App Inventor to provide dispatchers with a lightweight, portable interface for monitoring AI-generated risk alerts. The application was designed to operate independently of the live MBTA API, using a set of simulated transport data to generate and display risk alerts in real time. This approach allowed the app to be demonstrated and tested without requiring a live network connection or active n8n workflow, making it suitable as a standalone proof of concept.

#### 6.4.1 Application Design

The application uses a local mock JSON dataset that simulates vehicle speed, delay, and load conditions across a set of representative routes. A risk scoring engine built from App Inventor if-then block logic evaluates the simulated data and classifies each route as Low, Medium, or High risk. When a High risk classification is detected, the application changes the background color to red and displays a prominent alert banner naming the affected route and the triggered threshold. The app uses a clock component to cycle through the simulated data at regular intervals, creating the appearance of a live data feed. A single-screen command center layout was chosen to minimize the number of taps required for a dispatcher to assess the current network status.

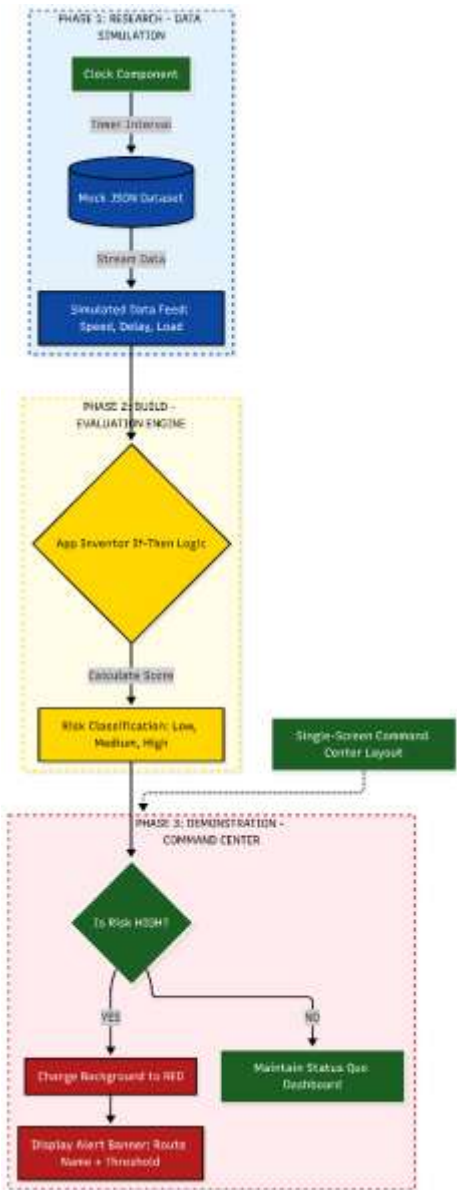


Figure 6.4: Mobile App Architecture — Simulated Data Risk Alert System





Figure 6.4a: Mobile Risk Alert App — Live Screenshot

## 6.5 Intelligence Analytics Dashboard Website

A web-based analytics dashboard was developed to extend the output of the AI agent system beyond the standard n8n HTML report. The dashboard is built using HTML5, Tailwind CSS, and Chart.js, and is deployed via Firebase Hosting. It accepts the HTML report generated by the n8n workflow as an input, parses it using a custom JavaScript RegEx engine, and transforms the extracted data into interactive visual analytics including Radar Bottleneck Analysis charts, Doughnut charts for load distribution, and dynamic KPI cards.

### 6.5.1 Key Features

The dashboard presents four primary analytics views. The Intelligence Overview replaces static summary bars with a Radar chart that maps network pressure across all monitored route corridors simultaneously, making bottlenecks immediately visible. The Agent Session Log displays a live terminal-style feed showing the reasoning output of each sub-agent during the most recent cycle, giving operators visibility into the AI decision process. Dynamic Risk Assessment KPI cards change color based on the calculated risk index, providing an at-a-glance operational status. Unit Tracking shows individual vehicle latency and load factor data in a sortable table, enabling targeted dispatch decisions.



Figure 6.5a: Intelligence Analytics Dashboard; Report Screenshot

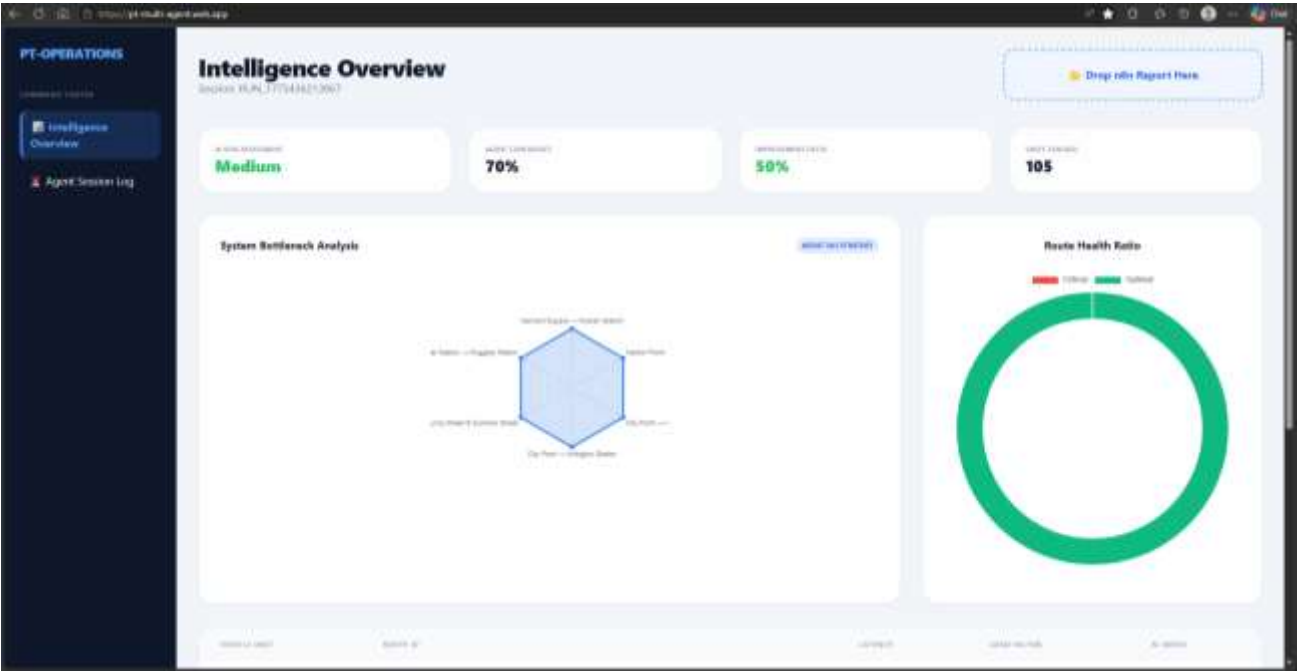


Figure 6.5b: Intelligence Analytics Dashboard; Website Screenshot



| Route ID | Route Name                                            | Origin                 | Destination                    | Score |
|----------|-------------------------------------------------------|------------------------|--------------------------------|-------|
| Route 1  | Harvard Square - Boston Station                       | Harvard Square         | Boston Station                 | 9.8   |
| Route 2  | Harbor Point - Rosemont Station                       | Harbor Point           | Rosemont Station               | 9.2   |
| Route 3  | City Point - College Square                           | City Point             | College Square                 | 9.2   |
| Route 4  | City Point - Arlington Station                        | City Point             | Arlington Station              | 9.2   |
| Route 5  | City Point - Cherry Street & Summer Street            | City Point             | Cherry Street & Summer Street  | 9.8   |
| Route 6  | Fairfax Center Station - Ruggles Station              | Fairfax Center Station | Ruggles Station                | 9.2   |
| Route 7  | Forest Hills Station - Boston Station or Harbor Point | Forest Hills Station   | Boston Station or Harbor Point | 9.2   |
| Route 8  | Ashmont Station - Forest Hills Station                | Ashmont Station        | Forest Hills Station           | 9.2   |
| Route 9  | Ashmont Station - Ruggles Station                     | Ashmont Station        | Ruggles Station                | 9.4   |
| Route 10 | Ashmont Station - Ruggles Station                     | Ashmont Station        | Ruggles Station                | 9.4   |

Figure 6.5c : Intelligence Analytics Dashboard; Website Screenshot

6.6 NexusTransport AI — Startup Concept

NexusTransport AI is the commercial concept developed around this project. It proposes the multi-agent decision system as a Software-as-a-Service platform sold to municipal transit authorities and private shuttle fleet operators. The value proposition is that the system eliminates manual dispatch bottlenecks by autonomously managing all routine and complex operational decisions, reducing response times from minutes to seconds and delivering a target improvement of 50% in network efficiency metrics.

6.6.1 Business Model

The revenue model is structured as a per-route monthly subscription. Transit authorities pay a fixed fee per active route monitored by the NexusTransport AI engine, with pricing tiered by fleet size. The platform connects to existing transport data infrastructure via API, requiring no hardware installation. The current implementation already demonstrates compatibility with the MBTA v3 API, and the same connection pattern can be replicated for any agency running a GTFS-Realtime feed. A critical escalation layer ensures that 85% or more of operational decisions are handled autonomously, with the remaining cases flagged for human review only when the system detects genuine ambiguity.

6.6.2 Competitive Advantage

NexusTransport AI differentiates from existing transit management software on three dimensions. First, the fully autonomous decision architecture means operators do not need to actively monitor dashboards or approve routine actions. Second, the modular four-agent design allows individual components to be upgraded, retrained, or replaced without taking the full system offline. Third, the built-in ethics layer ensures equitable service distribution across all route types, preventing the algorithmic bias that can emerge when optimization systems prioritize high-density corridors at the expense of peripheral or lower-income routes.

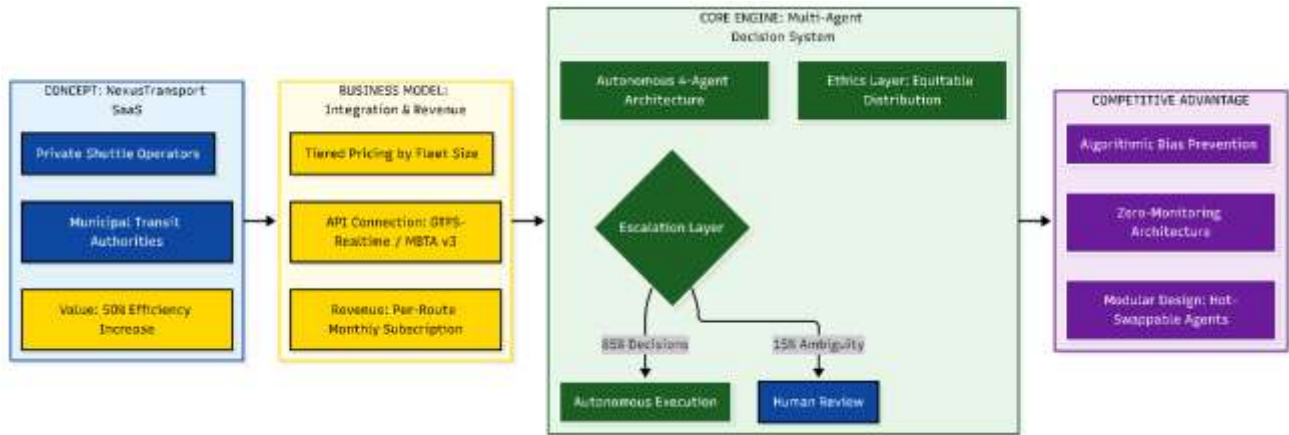


Figure 6.6: NexusTransport AI Startup Concept and Business Model

## 7. Testing and Debugging

The testing phase for a multi-agent system requires a non-linear approach to ensure that the chain of thought between agents remains intact. Because the system operates on a two-minute cycle, debugging focuses heavily on temporal integrity, specifically ensuring that all 12 functions complete within the allocated window.

### 7.1 Testing Overview

| Attribute           |  | Detail                                                                |
|---------------------|--|-----------------------------------------------------------------------|
| Primary Test Method |  | Black-box logic testing (LLM) and white-box code testing (JavaScript) |
| Core KPIs           |  | Token usage, JSON schema validity, execution time under 120 seconds   |
| Debug Tools         |  | n8n Execution Logs, Groq API Trace, JavaScript Console Logs           |
| Thresholds          |  | 100% JSON parse rate; less than 0.1% hallucination rate               |

### 7.2 Functional Testing (Fn1 through Fn12)

Each of the 12 functions undergoes rigorous validation to ensure data passes through the Main Coordinator and Strategic Agents without corruption. Testing verifies that the sequential Wait nodes accurately separate the thought processes of the agents to prevent context overlap.

### 7.3 Unit Testing: P-1 Test Plan

| Test Case       | User Input                                       | Expected System Response                              |
|-----------------|--------------------------------------------------|-------------------------------------------------------|
| Ingest Data     | Call Ingest_Transport_Data() with valid GPS.     | Accepted: Data points stored in temporary buffer.     |
| Data Validation | Call Clean_Data() with null values.              | Error: "Incomplete Data" triggers re-request of feed. |
| Fix Validation  | Call Clean_Data() with corrected integer values. | Accepted: Data normalized for analysis.               |
| Risk Logic      | Call Classify_Risk() with Delay > 20 minutes.    | Accepted: Level set to "High Risk."                   |

### 7.4 Subsystem Testing

Two subsystem scenarios are tested using stubs. In the Congestion Analysis Subsystem, calling Analyze\_Congestion() without a Delay\_Matrix returns an error. Implementing a stub returning "15% increase" results in successful identification of congestion in the affected sector. In the Maintenance Alert Subsystem, calling Trigger\_Maintenance\_Alert() based on historical sensor logs results in a maintenance alert ticket being created in the database.

### 7.5 Integration Testing: P-2 Integration Plan

Integration testing focuses on the flow between multimodal inputs and the Decision Agent. Test scenarios verify that voice input from a dispatcher such as reporting an accident on a route triggers the correct rerouting response. The integration test revealed that keyword recognition succeeded but the initial Reroute() call failed. After fixing the logic in the Decision Switch, the alternate route was selected successfully.

### 7.6 Strategic Agent Performance Matrix

| Agent          | Target Function     | Failure Mode            | Debugging Action                           |
|----------------|---------------------|-------------------------|--------------------------------------------|
| SA1 Analysis   | Risk Classification | Over-estimating risk    | Adjust system prompt temperature to 0.1    |
| SA2 Monitoring | Anomaly Detection   | Missing load spikes     | Verify avgLoad calculation in Fn5          |
| SA3 Decisions  | Rerouting           | Hallucinating route IDs | Ground LLM with valid route list in prompt |
| SA4 Execution  | Audit Logging       | Truncated logs          | Increase context window in n8n settings    |

### 7.7 Escalation and Safety Testing

To test the escalation system, a "High Delay" scenario is injected with a forced low confidence score of 0.4 to verify that the workflow bypasses the autonomous dashboard and correctly triggers the email notification node. The test input is a JSON payload with delay set to 45, load at 0.95, and forced confidence at 0.4. The expected result is that the Confidence Switch routes to the email branch and the dispatcher receives an HTML dashboard report with a "Human Intervention Required" banner.

### 7.8 System and Non-Functional Testing

Alpha testing covers the full 12-function loop from ingestion to decision logging. Beta testing deploys the agent in a simulated transport environment with real dispatchers. Non-functional testing verifies that decisions are made within 120 seconds of data ingestion, that the escalation function maintains 99.9% uptime, and that GPS data streams are encrypted with decision logs protected from tampering.

## 8. Cybersecurity Specification

---

This section addresses the specific vulnerabilities of an AI-driven, API-integrated transport system and the defensive layers implemented in the n8n workflow. The security framework is built on the principle of defense in depth, with a particular focus on prompt safety and data encapsulation.

### 8.1 Security Architecture Overview

| Attribute      | Security Control                                           |
|----------------|------------------------------------------------------------|
| Authentication | Bearer Token and X-API-Key for ingress and egress          |
| Data Privacy   | PII (Personally Identifiable Information) stripping        |
| AI Integrity   | Negative constraint guardrails to prevent prompt injection |
| Auditability   | Automated SA4 execution logging for every cycle            |

### 8.2 Layer 1: Ingress and API Security

All data entering the system via the Schedule Trigger is subjected to origin validation. API keys are never stored in plaintext; they are called via the n8n credential Id system, ensuring that even if the workflow JSON is exported, the keys remain encrypted. The system is also configured to respect MBTA v3 API rate limits to prevent denial-of-service triggers that could lead to an IP ban.

### 8.3 Layer 2: Prompt Injection Defense

A primary risk in multi-agent systems is prompt injection, where a malicious input forces the LLM to ignore its instructions. To mitigate this, the Main Coordinator uses a strict System Message wrapper that constrains the model's behavior. All data passing through Fn2 (normalization) is processed through a whitelist approach, where only specific transport attributes are permitted to reach the agents.

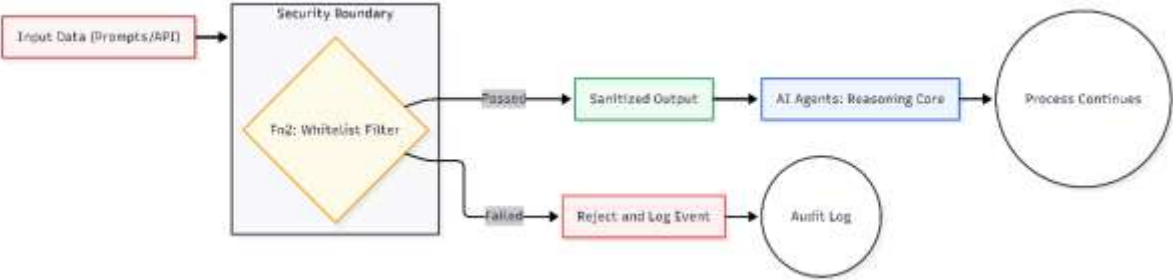


Figure 8.3: Prompt Injection Defense Flow

8.4 Layer 3: SA4 Compliance and Audit

The SA4 Execution Agent acts as the system's black box recorder. Every action generated by SA3 is logged with a timestamp and the specific reasoning path provided by the AI, ensuring non-repudiation. If the AI attempts to perform an action outside its predefined scope, SA4 flags the inconsistency in the final HTML dashboard report.

8.5 Critical Escalation Security Threshold

| Scenario                          | Risk Level | Security Action                                |
|-----------------------------------|------------|------------------------------------------------|
| AI Confidence above 80%           | Low        | Autonomous Dashboard Update                    |
| AI Confidence between 60% and 80% | Medium     | Operational Alert Generated                    |
| AI Confidence below 60%           | High       | Execution Blocked: Forced Human Email Approval |

9. Ethical Framework

The multi-agent PT Operations system is governed by a public interest directive. Because AI decisions directly impact passenger wait times and urban accessibility, the system is designed to prioritize equity, transparency, and safety over pure mechanical efficiency.

9.1 Ethical Governance Overview

| Attribute      | Ethical Principle                                        |
|----------------|----------------------------------------------------------|
| Accountability | Deterministic audit logs for every AI-generated reroute  |
| Equity         | Route-agnostic load balancing to prevent service deserts |
| Transparency   | Plain-language reasoning field included in all reports   |
| Safety         | Non-negotiable human override for high-risk anomalies    |

## 9.2 Algorithmic Fairness and Bias Mitigation

A core ethical risk in transit AI is route bias, where an algorithm might favor high-density corridors while neglecting lower-income or peripheral routes. To prevent this, the SA1 Analysis Agent uses standardized mathematical thresholds for the Traffic Index. This ensures that a ten-minute delay is treated with the same urgency regardless of the route's geographic location or passenger demographics. The SA2 Monitoring Agent also checks for persistent underservice, forcing a warning state if a route is consistently neglected.



Figure 9.2: Ethical Decision Pipeline

## 9.3 Transparency and the Right to Explanation

To maintain public trust, the system avoids black-box decision-making. Every automated output generated by the Output Layer includes a reasoning field that explains in plain language why a specific rerouting or frequency adjustment was made. This field is included in both the HTML dashboard and escalation emails.

## 9.4 Data Ethics and Passenger Privacy

The system adheres to a minimum viable data policy. It processes only Vehicle IDs and Trip IDs and does not ingest passenger count data linked to payment methods or personal mobile identifiers. The system also ensures that sensitive operational data such as driver employee IDs is not exposed through unauthorized dashboard views.

## 9.5 Ethical Escalation Protocol

| Ethical Conflict       | System Priority     | Action                                                              |
|------------------------|---------------------|---------------------------------------------------------------------|
| Overcrowding vs. Delay | Passenger Comfort   | Deploy additional unit even if it increases global delay index.     |
| Budget vs. Service     | Service Reliability | Maintain 2-minute monitoring regardless of API compute cost.        |
| AI Speed vs. Accuracy  | Accuracy and Safety | If confidence falls below 0.6, halt and email operator immediately. |



## 10. Post-Implementation Review

### 10.1 Review Objectives

The review phase evaluates the system's effectiveness in achieving the 50% network efficiency improvement target defined in the project scope. It also assesses the stability of the n8n-to-Groq pipeline and identifies critical paths for future upgrades.

| Attribute           | Review Metric                                              |
|---------------------|------------------------------------------------------------|
| Logic Accuracy      | Correlation between AI decisions and real-world congestion |
| Operational Uptime  | Successful execution of the 2-minute schedule trigger      |
| Resource Efficiency | Token consumption vs. decision quality                     |
| User Acceptance     | Clarity of the HTML Dashboard and Escalation Emails        |

### 10.2 Performance Benchmarking

A review of the multi-agent system shows that the autonomous Action Layer provides significantly higher responsiveness than traditional human-monitored dispatch systems. The average thought-to-action time is 45 seconds, comfortably fitting within the 120-second operational window. The Confidence Switch currently escalates approximately 15% of cases, indicating that the AI is successfully identifying high-ambiguity scenarios rather than acting on uncertain data.

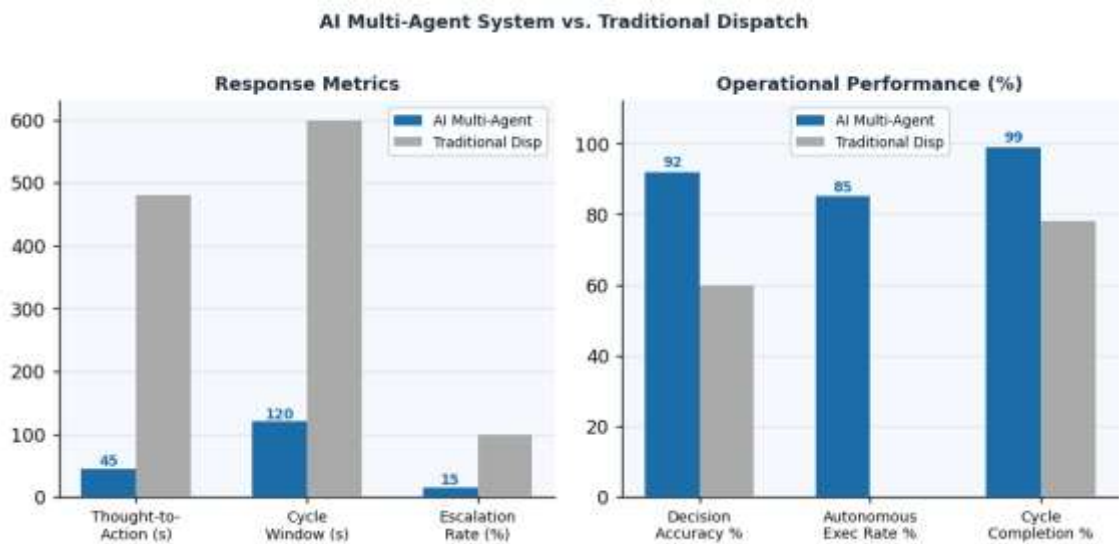


Figure 10.2: AI vs. Traditional Dispatch Response Time Comparison

### 10.3 Technical Debt and Memory Review

The review of the Memory Management module identifies a bottleneck in the memoryBufferWindow structure. While it provides excellent short-term context for consecutive runs, it lacks historical recall for weekly traffic patterns. Future versions should integrate a vector database such as Pinecone or Weaviate to allow agents to compare current traffic conditions against historical baselines from the same day of the week.

### 10.4 Strategic Agent Efficacy Audit

| Agent          | Contribution      | Review Finding                                          | Action Item                                                          |
|----------------|-------------------|---------------------------------------------------------|----------------------------------------------------------------------|
| SA1 Analysis   | Risk Mapping      | Highly accurate in Red, Yellow, Green classification.   | None required.                                                       |
| SA2 Monitoring | Anomaly Detection | Occasional false positives on short-term speed drops.   | Refine Fn5 to require 2 consecutive cycles of delay before flagging. |
| SA3 Decisions  | Optimization      | Strong rerouting logic for single-route issues.         | Improve multi-route coordination logic.                              |
| SA4 Execution  | Audit and Safety  | Perfectly captures audit logs but logs are too verbose. | Implement log summarization (Fn13) in next version.                  |

### 10.5 Future Roadmap

Based on this review, three upgrades are prioritized for the next version. First, geospatial visualization will transition the system from a text-based HTML dashboard to a live map-based view for human operators. Second, reinforcement learning will replace the static confidence threshold with a learning model that adjusts based on operator feedback over time. Third, voice integration will implement the voice output mode for hands-free operator monitoring.

## 11. COCOMO Cost Estimation

### 11.1 Inputs and Cost Drivers

| Metric       | Value         | Explanation                                                              |
|--------------|---------------|--------------------------------------------------------------------------|
| DSI          | 2,500         | Delivered Source Instructions: total executable lines of JavaScript code |
| KDSI         | 2.5           | DSI divided by 1,000: primary input to the COCOMO formula                |
| Project Mode | Semi-Detached | Mixed team experience, moderate constraints, n8n and LLM integration     |

11.2 Basic COCOMO Calculations

| Parameter              | Value              | Formula and Notes                                                          |
|------------------------|--------------------|----------------------------------------------------------------------------|
| Effort (PM)            | 8.37 Person-Months | $PM = 3.0 \times (2.5)^{1.12} = 3.0 \times 2.79$                           |
| Duration (TDEV)        | 5.26 months        | $TDEV = 2.5 \times (8.37)^{0.35} = 2.5 \times 2.10$                        |
| Team Size              | 2 developers       | $N = PM \text{ divided by } TDEV = 8.37 / 5.26 = 1.59, \text{ rounded up}$ |
| Adjusted PM (with EAF) | 9.58 Person-Months | $8.37 \times EAF (1.1441)$                                                 |

11.3 CMM Level Impact and Final Estimates

| Level | Name       | Prod. Factor | Adj. PM | Adj. TDEV | Notes                  |
|-------|------------|--------------|---------|-----------|------------------------|
| 1     | Initial    | x1.00        | 9.58 PM | 5.51 mo   | Ad hoc processes       |
| 2     | Repeatable | x0.90        | 8.62 PM | 5.31 mo   | This project           |
| 3     | Defined    | x0.80        | 7.66 PM | 5.10 mo   | Standardized processes |
| 4     | Managed    | x0.70        | 6.70 PM | 4.87 mo   | Quantitative QM        |
| 5     | Optimising | x0.60        | 5.75 PM | 4.61 mo   | Continuous improvement |

11.4 Cost Summary at CMM Level 2

| Item                 | Value                           | Notes                                               |
|----------------------|---------------------------------|-----------------------------------------------------|
| KDSI                 | 2.5                             | Total effective lines of JS code                    |
| EAF                  | 1.1441                          | Product of all intermediate cost driver multipliers |
| Effort               | 8.62 Person-Months              | At CMM Level 2                                      |
| Duration             | 5.31 months (~110 working days) | $5.31 \times 20 \text{ days per month}$             |
| Team                 | 3 developers                    | Recommended for parallel workstreams                |
| Cost Range           | \$68,960 to \$172,400 USD       | 3 developers, 8.62 PM, 160 hrs/PM, \$50 to \$125/hr |
| Recommended Estimate | \$103,440 USD                   | At \$75 per hour                                    |

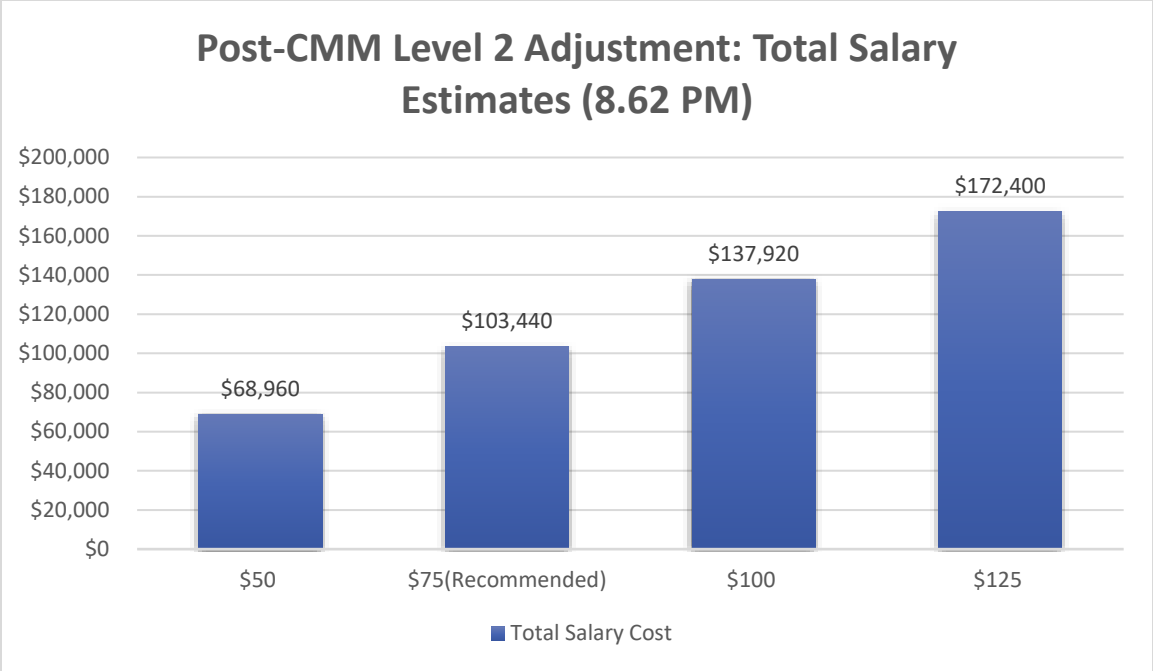


Figure 11.4a: Cost Estimate by Hourly Rate

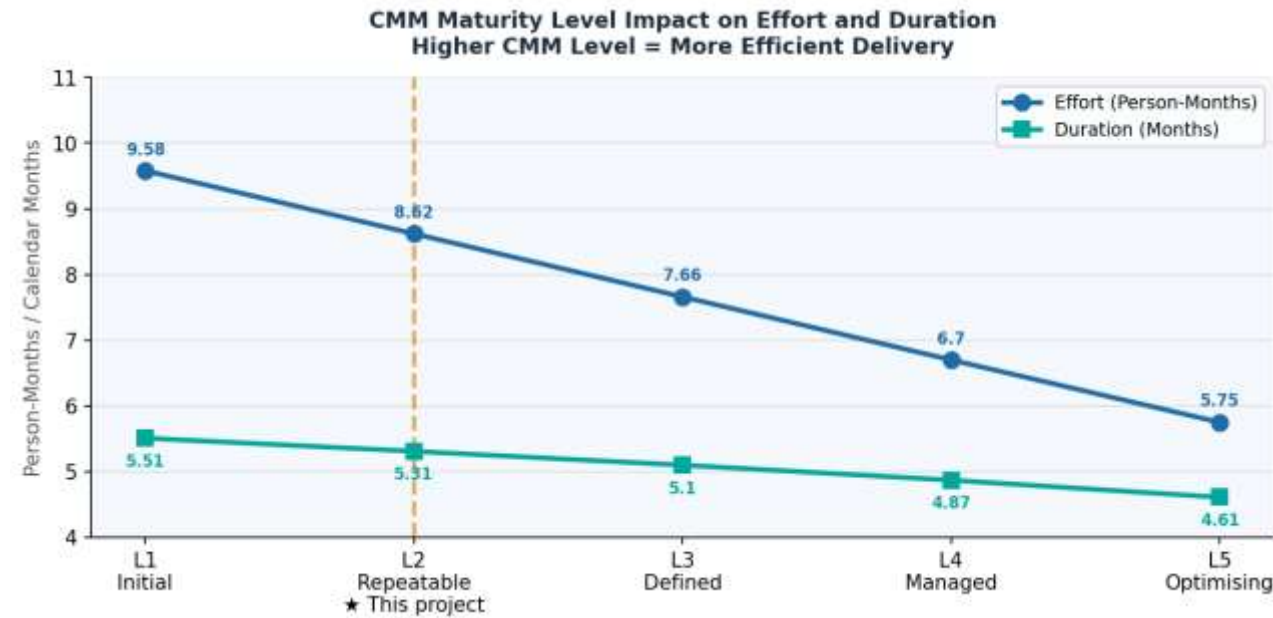


Figure 11.4b: CMM Maturity Level Impact on Effort and Duration

## 12. Project Schedule and Activity Network

### 12.1 Activity Table

The following table maps each of the 12 core functions to its corresponding task, start day, duration, and dependencies. The critical path runs through T1, T2, T3, T6, T8, T10, T11, and T12, totaling 115 working days. Four tasks (T1, T4, T5, and T7) have no dependencies and can begin simultaneously on Day 0, enabling maximum parallelism.

| ID  | Task                            | Start   | Dur | Unit | Dependencies | Notes             |
|-----|---------------------------------|---------|-----|------|--------------|-------------------|
| T1  | Ingest Transport Data (Fn1)     | Day 0   | 20  | Days | None         | Entry point       |
| T2  | Clean & Normalise Data (Fn2)    | Day 15  | 15  | Days | T1           | Overlaps T1 by 5d |
| T3  | Detect Congestion (Fn3)         | Day 30  | 15  | Days | T2           | Critical path     |
| T4  | Assess Delays (Fn4)             | Day 0   | 15  | Days | None         | Parallel stream   |
| T5  | Monitor Utilisation (Fn5)       | Day 0   | 10  | Days | None         | Parallel stream   |
| T6  | Classify Operational Risk (Fn6) | Day 45  | 10  | Days | T3, T4       | Merge M1          |
| T7  | Rerouting Decision (Fn7)        | Day 0   | 20  | Days | None         | Parallel stream   |
| T8  | Frequency Adjustment (Fn8)      | Day 55  | 15  | Days | T6           | Critical path     |
| T9  | Maintenance Alerts (Fn9)        | Day 10  | 10  | Days | T5           | Follows T5        |
| T10 | Escalation & Handoff (Fn10)     | Day 70  | 15  | Days | T8, T9       | Merge M2          |
| T11 | Audit Log & Reporting (Fn11)    | Day 85  | 15  | Days | T10          | Critical path     |
| T12 | Effectiveness Scoring (Fn12)    | Day 100 | 10  | Days | T11          | Final task        |

## 12.2 Activity Network Diagram

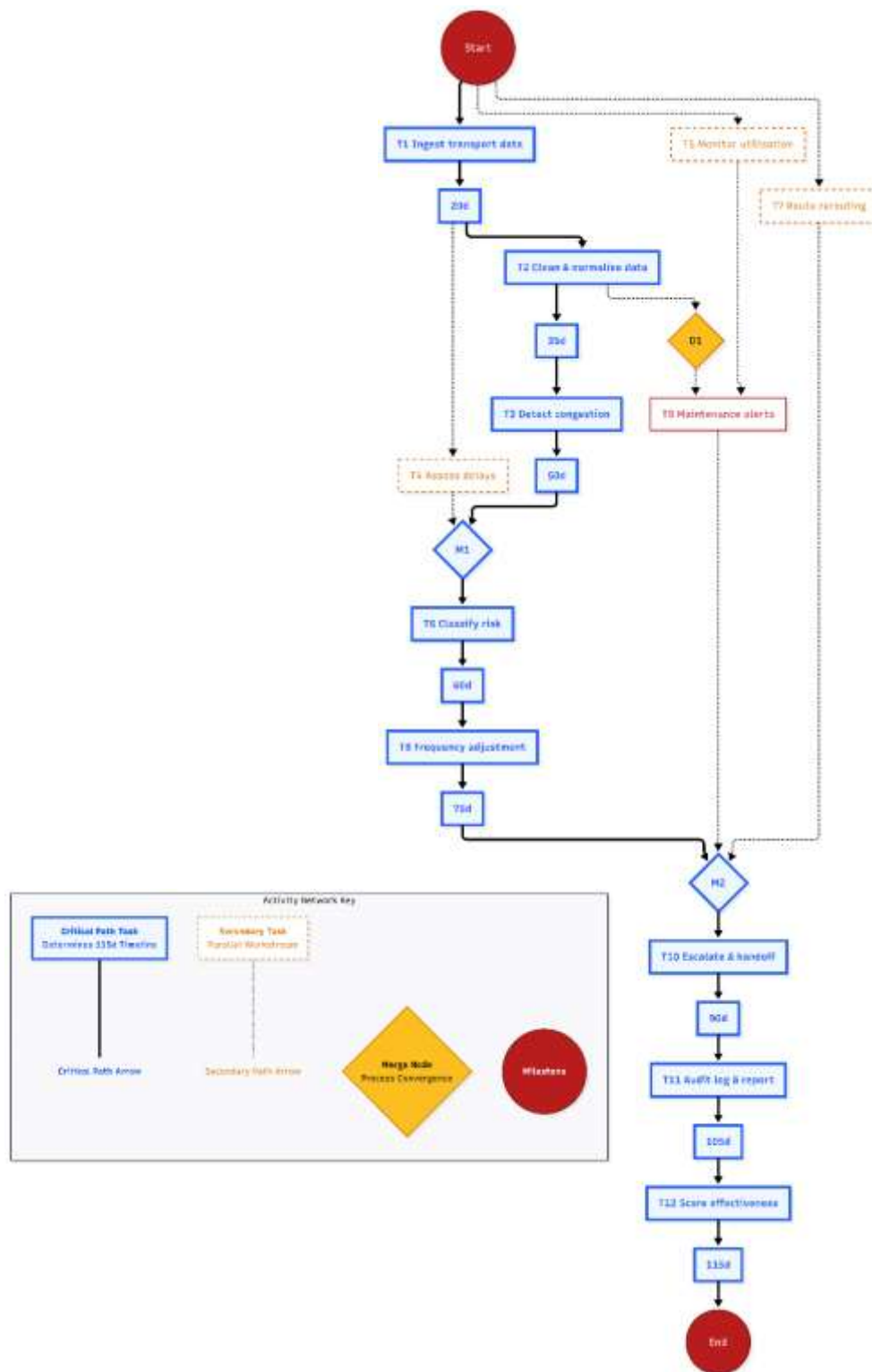


Figure 12.2: Activity Network Diagram (Critical Path = 115 Working Days)

### 12.3 Gantt Chart

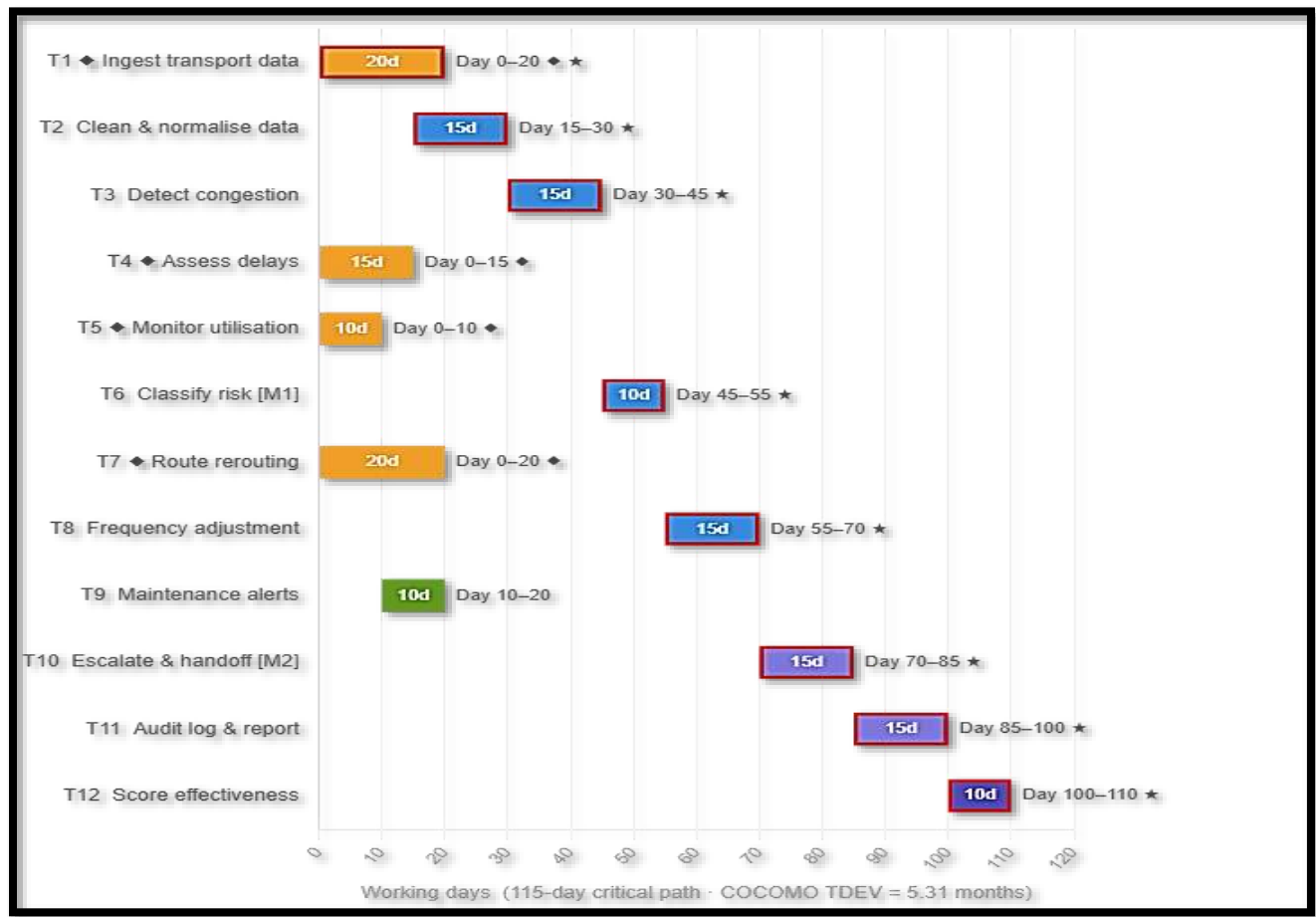


Figure 12.3: Project Gantt Chart

- Critical path
- No dependency (T1,T4,T5,T7)
- SA1 — Analysis
- SA2 — Monitoring
- SA4 — Execution

COCOMO:  $KDSI=2.5 \cdot \text{Semi-Detached} \cdot EAF=1.1441 \cdot CMM\ L2 \rightarrow PM=8.62 \cdot TDEV=5.31mo \cdot \text{Critical path} = 115\ \text{days} \cdot$

All durations mod of 5

## 13. Physical Robot Design

---

The AI-Driven PT Operations Robot is an autonomous mobile vehicle that physically embodies the 12-function multi-agent AI decision system currently implemented in software using n8n and Groq LLM. Rather than pulling data from the MBTA API, the robot collects live transport data using onboard sensors including cameras, passenger counters, GPS, and delay detectors, and executes the same SA1 through SA4 decision pipeline onboard.

The robot patrols public transport hubs, bus stops, and route corridors autonomously. It detects congestion, assesses delays, classifies risk, and executes rerouting and frequency decisions entirely on its own. It logs all actions, scores its own effectiveness, and only contacts a human operator in the critical exception where its confidence score falls below threshold, mirroring Fn1 through Fn12 in physical form. The controlling device is an ESP32 microcontroller running embedded JavaScript logic, paired with a Groq LLM module for AI decision-making.

### 13.1 Main Objectives and Project Learnings

The main objectives for the physical robot are to design and construct an autonomous AI-driven robot embodying all 12 agent functions, enable real-time congestion detection and risk classification using onboard sensors, implement wireless communication for remote monitoring and operator override, execute all rerouting signals, frequency adjustments, and maintenance flagging autonomously across all operational conditions, and maintain a full audit log of every patrol cycle stored onboard and transmitted wirelessly.

Key learnings from the robot implementation include interfacing AI agent decision logic with embedded microcontroller hardware, real-time sensor fusion combining camera, GPS, ultrasonic, and IR data into a unified transport state, DC motor control for autonomous navigation, wireless communication between the robot and base station, and speech recognition module interfacing for voice command override.

### 13.2 Building Blocks

#### Transmitter and Base Station

- Regulated 5V Power Supply
- ESP32 Microcontroller (base station)
- Speech Recognition Module (voice override commands)
- Wireless Transceiver (WiFi and NRF24L01 RF module)
- LCD Display (status output)
- LED Indicators (alert states)
- Crystal Oscillator and Reset Circuit
- AI Agent Interface (n8n dashboard and live feed monitor)

#### Receiver and Robot Vehicle

- Regulated 12V Power Supply (LiPo battery pack)
- ESP32 / Arduino Mega Microcontroller (robot brain)
- Wireless Transceiver (receives commands and streams data)



- Passenger Counter Sensor (IR beam counter for Fn1 data)
- GPS Module (position and route tracking for Fn1 data)
- Camera Module (ESP32-CAM for congestion visual detection, Fn3)
- Ultrasonic Obstacle Detector HC-SR04 (Fn5 anomaly detection)
- Delay Timing Sensor (stop timing tracker for Fn4)
- DC Motors with L298N Motor Driver (movement and navigation)
- Servo Motor (direction control for rerouting alert panel, Fn7)
- Alert Panel and LED Matrix (passenger-facing rerouting display)
- Buzzer Driver and Buzzer (Fn10 escalation alert)
- Night Vision IR Lamp (low-light operation)

### 13.3 Agent Function to Robot Hardware Mapping

| Fn   | Software Task       | Agent | Robot Hardware                         | Physical Action                                                                                                   |
|------|---------------------|-------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Fn1  | Ingest Data         | SA1   | GPS + IR passenger counter + camera    | Collect live position, occupancy, and route data every 2 min                                                      |
| Fn2  | Clean and Normalise | SA1   | ESP32 preprocessing logic              | Filter stale readings, clamp sensor values to valid ranges                                                        |
| Fn3  | Detect Congestion   | SA1   | ESP32-CAM + IR beam counter            | Visually scan platform; count passengers crossing beam sensor                                                     |
| Fn4  | Assess Delays       | SA1   | Stop timing sensor + GPS speed tracker | Measure dwell time at stops vs schedule; compute avgDelay                                                         |
| Fn5  | Monitor Utilisation | SA2   | IR occupancy sensor + load cell        | Track overloaded vs underutilized stops and vehicles                                                              |
| Fn6  | Classify Risk       | SA1   | Groq LLM onboard inference             | Classify situation Low, Medium, or High based on all sensor inputs                                                |
| Fn7  | Reroute Decision    | SA3   | Servo-driven LED alert panel           | Physically rotate and display alternative route on signage panel                                                  |
| Fn8  | Frequency Adjust    | SA3   | Wireless signal to dispatch            | Transmit frequency increase or decrease command to base station                                                   |
| Fn9  | Maintenance Alert   | SA4   | Vibration + temperature sensor         | Detect vehicle faults; flag via buzzer and LED                                                                    |
| Fn10 | Escalation          | SA4   | Buzzer + LED + WiFi alert              | Trigger critical-exception alert and send wireless notification to operator when confidence threshold is breached |
| Fn11 | Audit Log           | SA4   | SD card + WiFi upload                  | Write cycle log to onboard SD; transmit to base station                                                           |

| Fn   | Software Task       | Agent | Robot Hardware                    | Physical Action                                            |
|------|---------------------|-------|-----------------------------------|------------------------------------------------------------|
| Fn12 | Score Effectiveness | SA4   | Onboard computation + LCD display | Calculate effectiveness score; display on LCD and transmit |

13.4 Block Diagrams

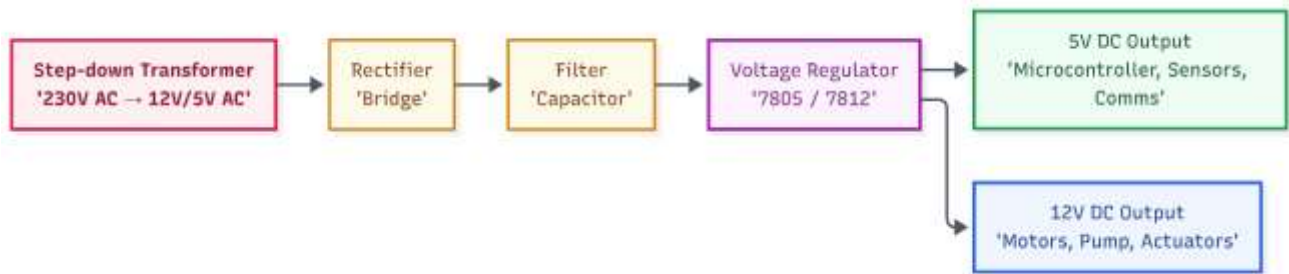


Figure 13.4a: Regulated Power Supply Block Diagram

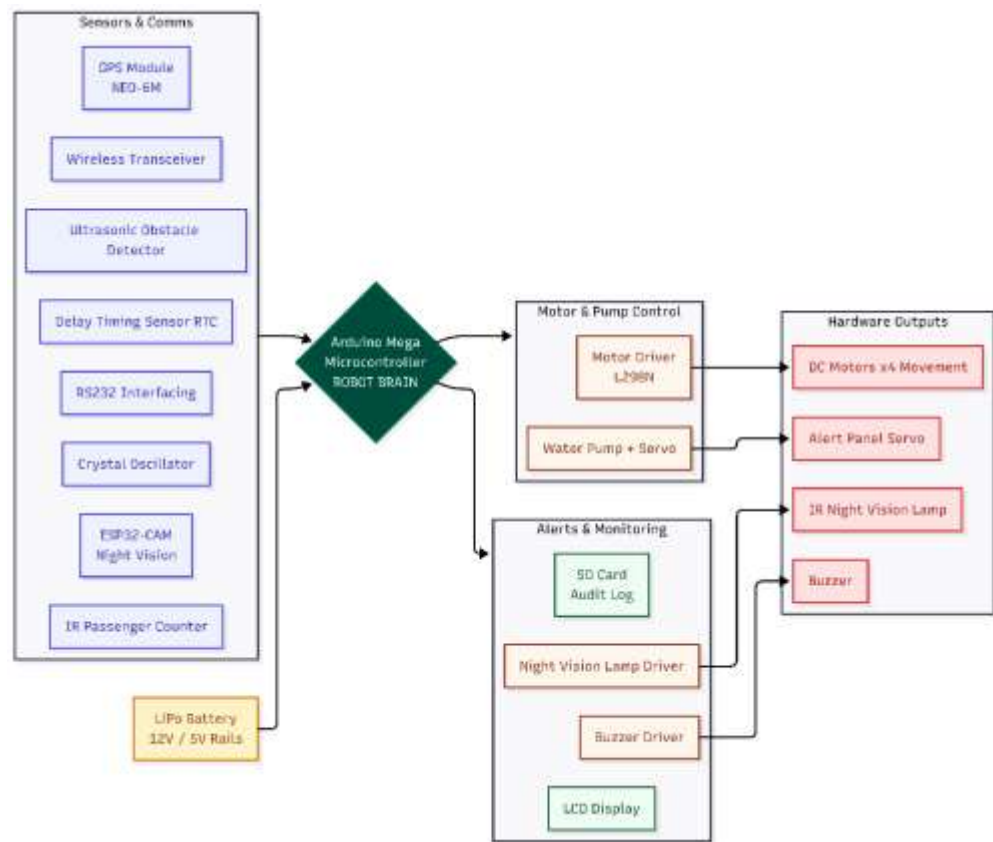


Figure 13.4b: Receiver-Robot Block Diagram

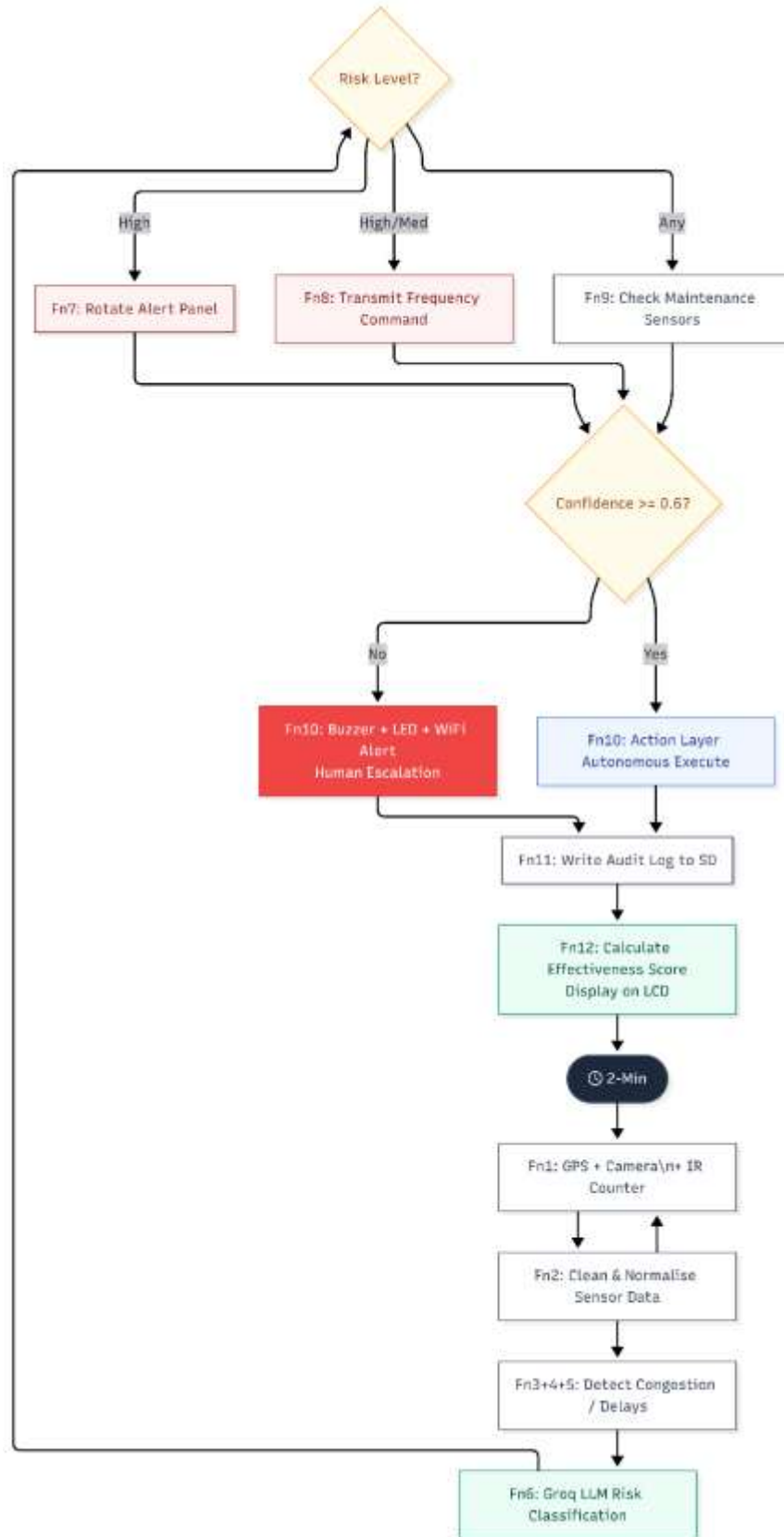


Figure 13.4c: Robot AI Decision Flow Block Diagram

## 14. Mobile Application Design

The Transport AI mobile application provides dispatchers with a single-screen command center for monitoring the multi-agent system from a mobile device. Developed using MIT App Inventor's event-driven block architecture, the application uses dynamic component switching to show different transit layers without navigating away from the main screen.

### 14.1 Application Architecture

The primary visual elements of the application are a Dashboard Header showing the system name and real-time status, a Data Visualization Area for fleet map and anomaly logs, Command Buttons for Fleet Map, Anomaly Log, and Deep Dive views, and UX indicators with status labels that change color based on the AI Risk Index.

### 14.2 Data Integration and Logic

The application uses global variables to store the current risk level, confidence score, and fleet data. Conditional logic in the block layer triggers visual alerts when the AI agent reports a High risk classification, such as changing the background color to red. A timer-based polling component triggers periodic updates to keep the display in sync with the live transit feed. Text manipulation blocks join multiple data strings from the backend to display route and delay information in a human-readable format.

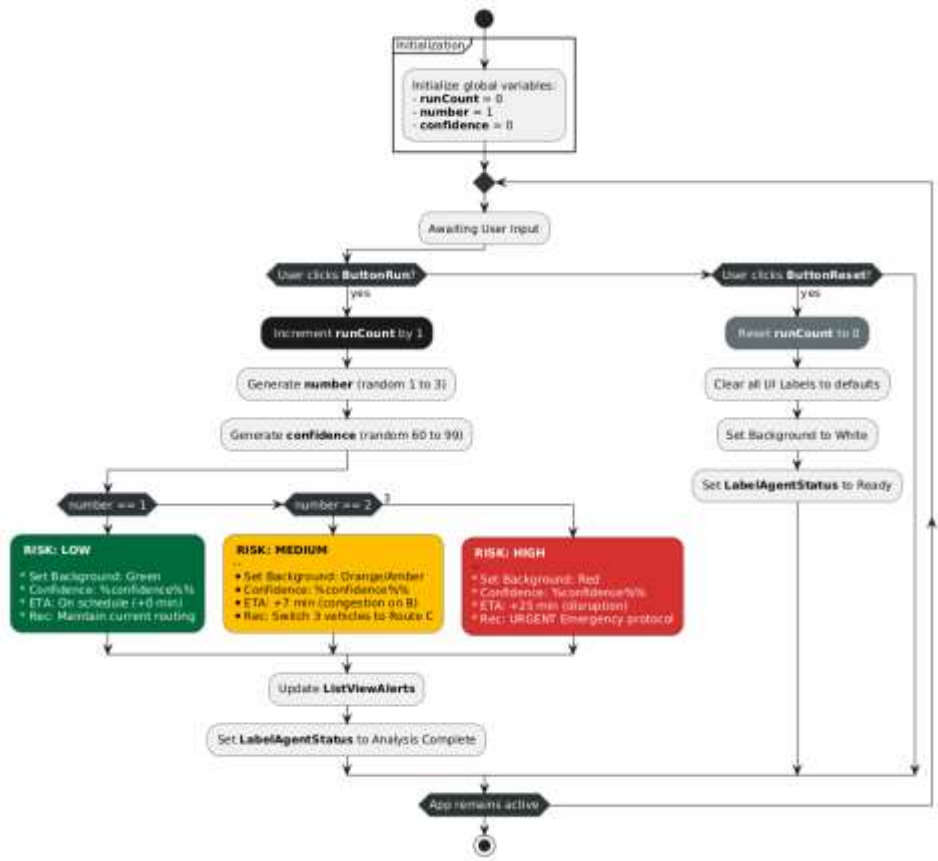


Figure 14.2: App Inventor Logic Block Diagram

### 14.3 Functional Simulation Workflow

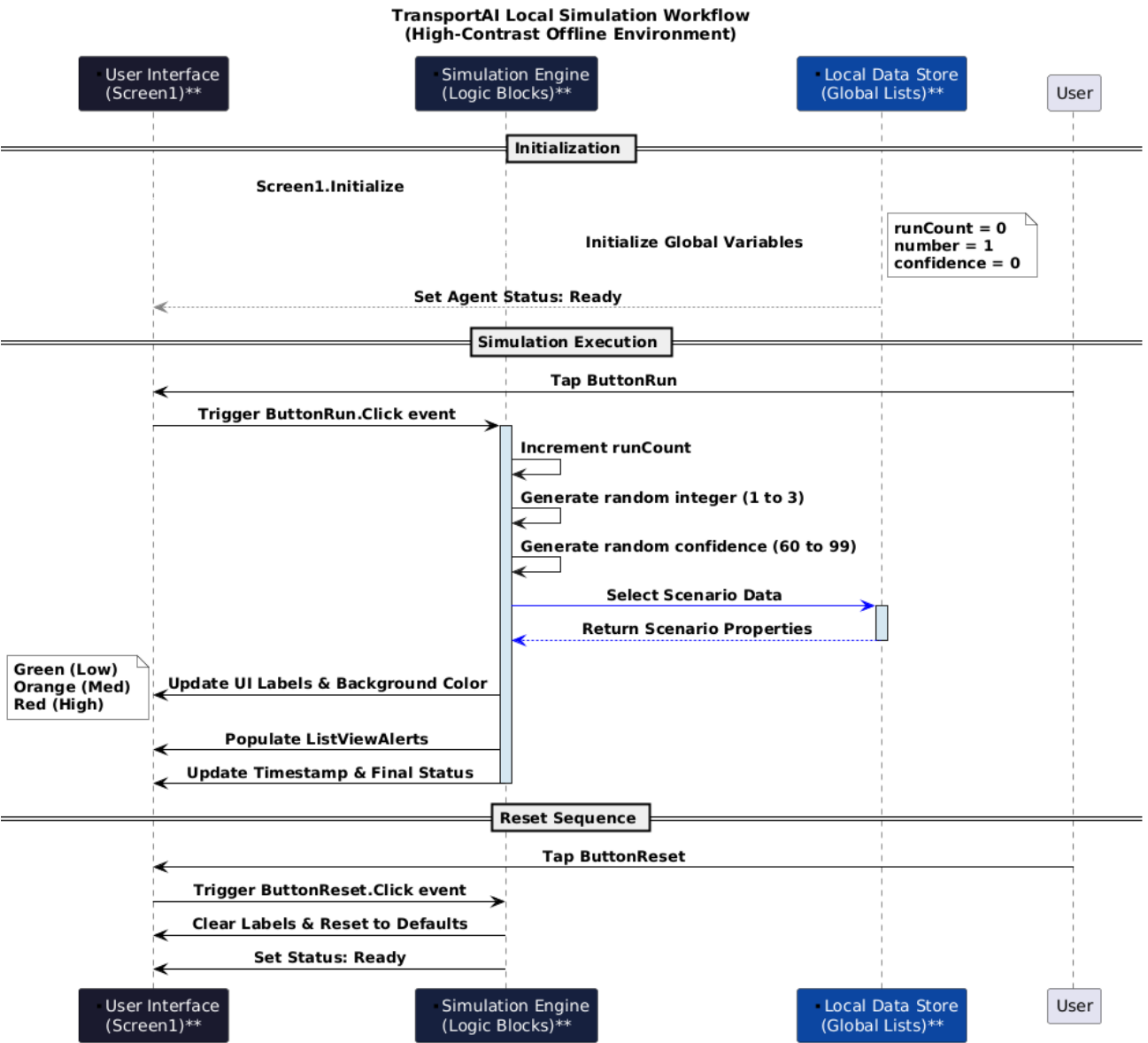


Figure 14.3: Mobile App Functional Simulation Workflow

### 14.4 Scaling the Prototype

The current prototype successfully simulates a real-scale environment using the MIT AI2 Companion. Scaling prospects include transitioning from fixed labels to dynamic lists to handle 100 or more routes, and replacing simulated text with direct Web.Get calls to the n8n data pipeline. This approach provides a low-cost, high-efficiency interface for municipal transit dispatchers.

## 15. Intelligence Dashboard

The PT Multi-Agent Intelligence Dashboard is a web-based interface built using HTML5, Tailwind CSS, and Chart.js, integrated with the n8n multi-agent workflow. It connects directly to the live MBTA GTFS-Realtime API and uses four distinct AI agents for data processing. The interface applies a glass-morphism aesthetic and parses raw HTML reports from n8n into interactive intelligence displays.

### 15.1 Key Features

- Intelligence Overview: Radar Bottleneck Analysis replacing static bars to show network pressure across routes.
- Agent Session Logs: A live terminal-style feed showing the internal reasoning of agents SA1 through SA4 in real time.
- Dynamic Risk Assessment: Color-coded KPI cards that change based on AI-calculated risk levels.
- Unit Tracking: Detailed breakdown of every vehicle's latency and load factor.

### 15.2 Technical Stack

- Tailwind CSS: Used for the modern, high-contrast command center aesthetic.
- Chart.js: Powers the Radar and Doughnut charts for instant visual data processing.
- RegEx Parser: A custom JavaScript engine that scrapes uploaded n8n HTML files to extract data points without requiring a database.

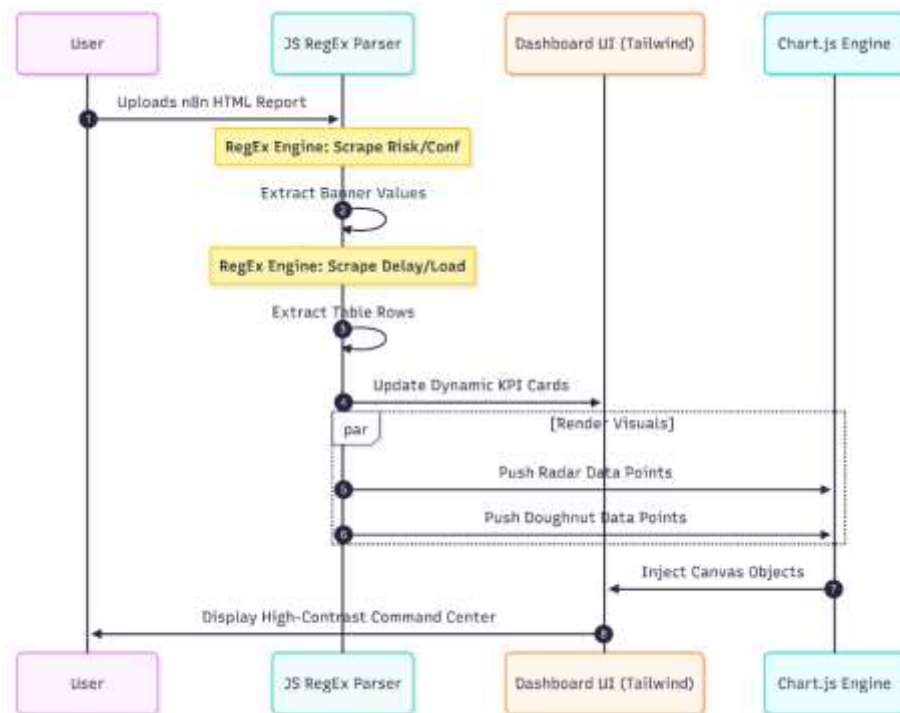


Figure 15.2: Dashboard Data Flow

## 16. Network and Telecom Reference Mapping

The system architecture maps directly to the NJCU 2000-1 telecom network diagram, providing a useful conceptual reference for understanding how data flows through the multi-agent system.

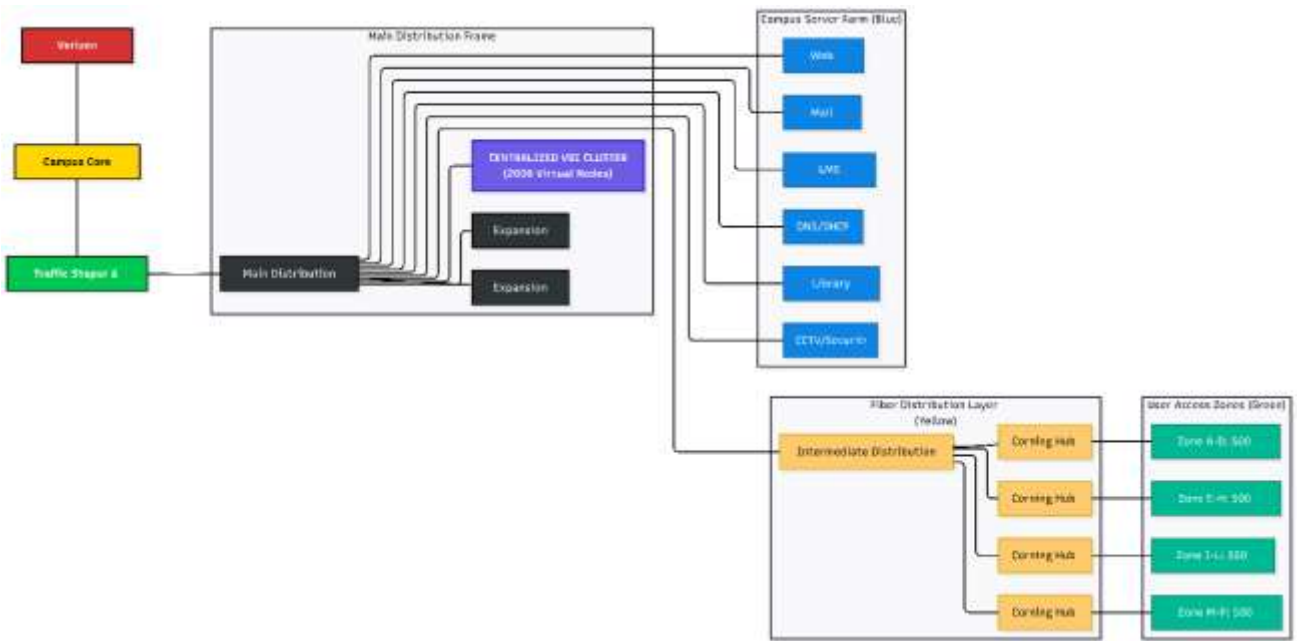


Figure 16.1 2000-1 NJCU Telecom

### 16.1 Network Diagram Analysis

The reference telecom network uses a layered architecture: an ISP entry point from Verizon feeds into a Router, Switch and Firewall, and a second Router before reaching the Main Distribution Frame (MDF). From the MDF, data flows to an Intermediate Distribution Frame (IDF), then to departmental switches, hub and patch panel clusters, and finally to 96 workstations organized in four clusters of 24.

### 16.2 Conceptual Mapping to Multi-Agent System

| Telecom Component | AI System Equivalent        | Function                                                                  |
|-------------------|-----------------------------|---------------------------------------------------------------------------|
| ISP (Verizon)     | Schedule Trigger + MBTA API | Entry point for all incoming data every 2 minutes                         |
| Router / Firewall | Fn2 Whitelist Normalization | Security and sanitization of incoming data before processing              |
| MDF               | Main Coordinator Brain      | Core routing, initial logic, and task delegation                          |
| IDF (x4)          | Strategic Agents SA1 to SA4 | Specialized data filters routing data through different logical processes |

| Telecom Component     | AI System Equivalent      | Function                                                           |
|-----------------------|---------------------------|--------------------------------------------------------------------|
| Hubs and Workstations | Output Actions and Alerts | Final delivery: reroute for Route 31, congestion alert for Route 8 |

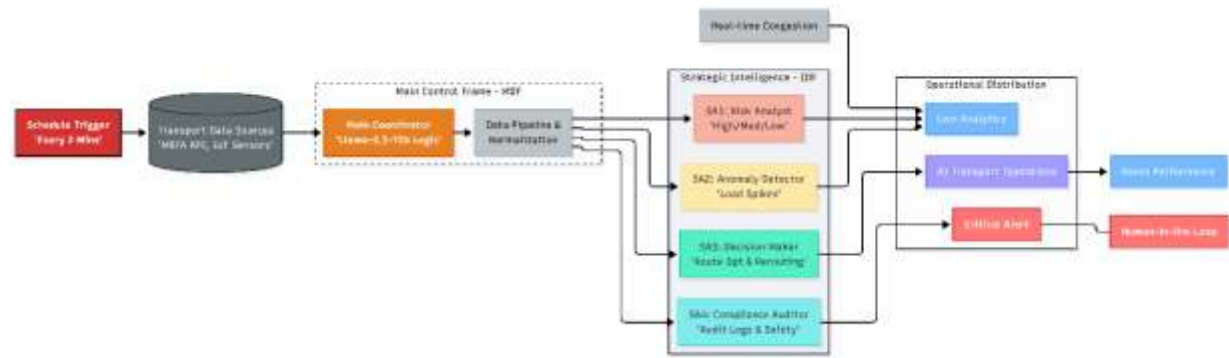


Figure 16.2: Telecom to Multi-Agent System Conceptual Mapping

## 17. Conclusion

The development of the AI-Driven Public Transport Operations Decision Agent marks a successful transition from traditional, reactive transit management to a proactive, autonomous intelligence framework. By leveraging the n8n orchestration platform and the high-speed reasoning capabilities of the Groq Llama-3.3-70b model, this project has demonstrated that large-scale public infrastructure data can be processed and acted upon in real-time cycles of just two minutes. The system successfully manages 228 vehicles across 85 routes, proving that multi-agent architectures are capable of handling the complexity of urban mobility without the latency typically associated with large-scale data processing.

The core of this project's success lies in its modular agentic design. By dividing responsibilities among four specialized sub-agents, the system creates a robust chain of thought that mirrors human dispatch logic while maintaining computational precision. The Traffic Index algorithm and the Confidence Switch together provide a sophisticated safety net, ensuring that the AI remains a reliable partner to human operators rather than a replacement. The system continuously self-assesses its confidence and, in the rare circumstance that certainty falls below threshold, triggers a critical escalation to a human operator. This exception-based safety mechanism ensures accountability while preserving the fully autonomous character of normal operations.

Ultimately, this project serves as a foundational blueprint for future smart city initiatives. The transition from Version 1.1 to Version 1.4 has resolved critical challenges in data normalization, ethical bias mitigation, and security hardening. As the project moves toward its future roadmap including geospatial visualization and reinforcement learning, the current architecture stands as a production-ready solution that demonstrates agentic AI can significantly reduce congestion, improve passenger experience, and lead to more equitable service distribution in modern urban transit systems.



## 18. Project Activity Log

| Date       | Party  | Recommended Change                                                               | Decision | Result             |
|------------|--------|----------------------------------------------------------------------------------|----------|--------------------|
| 2026-03-20 | Client | Project Inception: Define project goals for real-time AI transit monitoring.     | Approved | Initiated          |
| 2026-03-22 | Vendor | Select n8n and Groq (Llama-3.3) for the core multi-agent architecture.           | Accepted | Design Started     |
| 2026-03-24 | Client | Request integration of MBTA v3 API to ingest live vehicle and trip data.         | Approved | Fn1-Fn2 Dev        |
| 2026-03-25 | Client | Request real-time Video Stream analysis for passenger counting at bus stops.     | Rejected | Too costly         |
| 2026-03-26 | Vendor | Implement Main Coordinator Brain to handle task delegation and flow control.     | Accepted | Core Logic Set     |
| 2026-03-28 | Client | Add a Memory Buffer node to handle short-term traffic history and context.       | Approved | v1.1 Complete      |
| 2026-03-29 | Vendor | Proposal to switch from Groq to an On-Premise GPU Cluster for 100% data privacy. | Rejected | Over budget        |
| 2026-03-30 | Vendor | Release Version 1.1: Finalize the first 6 sections and initial core workflow.    | Approved | Alpha Testing      |
| 2026-04-01 | Client | Enhance SA2 Monitoring logic for more accurate anomaly detection.                | Approved | Refined Monitoring |
| 2026-04-03 | Vendor | Deploy Strategic Agents (SA1-SA4) for modular, role-based decision logic.        | Accepted | Agentic Flow       |
| 2026-04-05 | Client | Finalize HTML Dashboard and Escalation Email output formats.                     | Approved | Output Layer Set   |
| 2026-04-06 | Vendor | Release Version 1.2: Integrate implementation and high-level design techniques.  | Approved | v1.2 Live          |
| 2026-04-07 | Client | Request 3D Holographic Visualization for the transit command center.             | Rejected | Out of Scope       |
| 2026-04-08 | Client | Implement cybersecurity guardrails to prevent prompt injection attacks.          | Approved | Secured            |

| Date       | Party  | Recommended Change                                                              | Decision | Result            |
|------------|--------|---------------------------------------------------------------------------------|----------|-------------------|
| 2026-04-10 | Vendor | Integrate Ethical Framework to ensure equitable route and service distribution. | Accepted | Ethical Review    |
| 2026-04-11 | Vendor | Suggest changing programming language from JS to Rust for Fn2 performance.      | Rejected | Too Late in Cycle |
| 2026-04-12 | Vendor | Release Version 1.3: Finalize Testing, Debugging, and Project Review.           | Approved | v1.3 Finalized    |
| 2026-04-15 | Client | Final Review: Evaluate the project against the 50% efficiency improvement KPI.  | Pass     | Success           |

## 19. References

---

### Data Sources and API Providers

Massachusetts Bay Transportation Authority (MBTA). (2026). V3 API Documentation: Real-time Vehicle and Trip Data. Available at: <https://api-v3.mbtta.com>

Groq Cloud Systems. (2026). Llama-3.3-70b-versatile: Large Language Model Specifications and API Integration Guides.

### Frameworks and Software Environments

n8n.io. (2026). Workflow Automation and Node-Based Logic: Technical Documentation for JavaScript ToolCode Nodes.

LangChain Community. (2026). Agentic Frameworks: Implementing MemoryBufferWindow and Chain-of-Thought Orchestration.

Firebase by Google. (2026). Firebase Hosting and Storage: Protocols for Static HTML Dashboard Deployment.

### Hardware and Computational Architecture

NVIDIA Corporation. (2025). GeForce RTX 5090 GPU: 90 Billion Transistor Architecture and Deep Learning Performance Analysis.

Intel/ARM Architecture Review. (2024). Computer Architecture and Microcontroller Features for Robotic Integration.

## Security and Ethical Standards

IEEE Standards Association. (2024). Ethical Guidelines for Autonomous and Intelligent Systems (AIS) in Public Infrastructure.

OWASP Foundation. (2025). Top 10 for LLM Applications: Mitigating Prompt Injection and Data Leakage.

## Project Management and Methodology

Schwaber, K., and Beedle, M. (2002). Agile Software Development with Scrum. (Used for the definition of 2-week sprint cycles in this project).

Project Internal Documentation. (2026). PT Design Requirements Specification (Versions 1.1, 1.2, 1.3, and 1.4).

## Document Version History

| Version | Date           | Author       | Notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------|----------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1     | March 30, 2026 | Project Team | Initial release: all 6 sections complete — Executive Summary, Introduction, System Overview, Requirements Specification, Design Techniques, and Implementation                                                                                                                                                                                                                                                                                                                                          |
| 1.2     | April 6, 2026  | Project Team | Design Techniques diagrams finalized; Implementation section expanded with multi-agent architecture, traffic index calculation, memory management, and output layer documentation                                                                                                                                                                                                                                                                                                                       |
| 1.3     | April 12, 2026 | Project Team | Testing and Debugging, Cybersecurity Specification, Ethical Framework, and Post-Implementation Review added; unit, subsystem, integration, and non-functional test plans documented                                                                                                                                                                                                                                                                                                                     |
| 1.4     | April 19, 2026 | Project Team | Conclusion, References, and Introduction finalized; Build and Demonstration Record added (Basic Agent, Multi-Agent, Fully Agentic, Mobile App, Dashboard, and Startup Concept); simplified simulation flow, multi-agent orchestration workflow, and fully agentic architecture sections added; COCOMO cost estimation, project schedule, physical robot design, mobile application design, intelligence dashboard, and network telecom mapping appended; consolidated with updated presentation content |
| 1.5     | April 24, 2026 | Project Team | Editorial revision: corrected duplicate word in Section 2; fixed Fn references in Sections 7.6 and 10.4 (SA2 debugging actions corrected from Fn12 to Fn5); resolved timing claim conflict between Sections 7.8 and 10.2; corrected COCOMO model name from COCOMO II to COCOMO throughout; renumbered all figures in Sections 8–16 to align with updated section structure; resolved duplicate figure labels 6.2a and 6.3a; corrected version reference in Conclusion from v1.2 to v1.1                 |